Supplementary Material

Counting Collapse: A Formally Verified Attractor Theory of Repetition Hallucination in Autoregressive Text-to-Speech

This document accompanies the 4-page ICASSP 2026 submission and is self-contained. It reproduces the formal artifact in full, states precisely what it does and does not certify, documents the benchmark and measurement protocol down to the level a re-implementer needs, reports the negative result of Section 4.4 of the main paper in the detail it deserves, and answers, with new measurements rather than argument, five alternative explanations for the main results raised across two rounds of review (Section 8). Every claim below is traceable to a specific file in the project repository, cited inline by path.

Contents

1 The formal development in full **2**
- 1.1 `CountingCollapse.lean` . . . 3
- 1.2 `AttentionDilution.lean` . . . 8
- 1.3 The axiom audit . . . 12

2 What is deliberately not formalized **13**
- 2.1 What the remark actually claims . . . 14
- 2.2 Why this was not formalised . . . 14
- 2.3 What a full treatment would need . . . 15

3 The benchmark in full **16**
- 3.1 The k-ladder . . . 16
- 3.2 Word-repetition carrier templates . . . 16
- 3.3 Number phrases . . . 17
- 3.4 Sentence repetition and tongue twisters . . . 18
- 3.5 The matched-control construction . . . 18
- 3.6 The `boundary_units` field . . . 19
- 3.7 The instrumented subset . . . 19

4 Measurement protocol in full **20**
- 4.1 The conjunctive correctness criterion . . . 20
- 4.2 The outcome taxonomy . . . 20
- 4.3 Audio degeneracy detectors . . . 21
- 4.4 Effective rank, spectral slope, and Kirchhoff index . . . 22
- 4.5 The template bootstrap . . . 22

5 The negative result, in detail **23**
5.1 What the boundary-difference estimator was . . . 23
5.2 Why it was the natural first choice . . . 24
5.3 The measurement that killed it . . . 24
5.4 Why the failure is self-defeating by construction . . . 24
5.5 The replacement: boundary-free dispersion . . . 25

6 Per-model implementation notes **25**
6.1 Llasa (1B / 3B / 8B) . . . 26
6.2 XTTS-v2 . . . 27
6.3 Qwen3-TTS (0.6B / 1.7B) . . . 27

7 Reproducibility **28**
7.1 The four conda environments . . . 28
7.2 Other pitfalls documented in the README . . . 28
7.3 Exact commands . . . 29
7.4 The verification gate . . . 30

8 Answering the alternatives **30**
8.1 Naturalness: is the dissociation just improbability? . . . 31
8.2 Unit invariance: repetitions, or acoustic tokens? . . . 32
8.3 Capacity confound: is effective-rank decline tautological? . . . 34
8.4 The competing account: does the count survive in the representation? . . . 35
8.5 The XTTS repetition-penalty ablation . . . 37
8.6 Statistical treatment . . . 38

1 The formal development in full

The formal artifact lives at `lean/SpectralTTS/` and is organised into two files: `SpectralTTS/CountingCollapse.lean` (Theorem A and its supporting lemmas) and `SpectralTTS/AttentionDilution.lean` (Lemma B). Both import all of `Mathlib` and are checked against `leanprover/lean4:v4.34.0-rc1` with `mathlib` pinned at the same tag (`lean/SpectralTTS/lean-toolchain`, `lean/SpectralTTS/lakefile.toml`).

Typesetting convention. The listings below reproduce the two Lean source files exactly: every identifier, every tactic, every line of code and every word of every doc-comment is preserved verbatim and in its original order. The one substitution made for typesetting is that Lean's Unicode mathematical notation is rendered through the corresponding standard LaTeX math macro, because the base monospace font used in this document does not contain those glyphs and raw Unicode would silently disappear from the printed page (we checked this directly before committing to this approach). Concretely, in the listings that follow: $\forall$ stands for Lean's universal quantifier, $\exists$ for the existential quantifier, $\leq$ for the order relation, $\to$ for the function-space/implication arrow, $\in$ for set/type membership, $\sum$ for the finite-sum big operator, $\mathbb{R}$ and $\mathbb{N}$ for the real and natural number types, $\leftarrow$ for Lean's "rewrite backwards" arrow, and $\langle \cdot, \cdot \rangle$ for Lean's anonymous-constructor brackets; δ and ϵ stand for the one-letter Greek identifiers used throughout both files. Separately, mathlib's naming convention of suffixing a lemma name with a subscript zero to mark a positivity/non-degeneracy side condition (rendered here, e.g., as `div_le_iff0`) is printed with a

plain digit rather than a Unicode subscript, for the same font reason. No token is added, removed, or reordered.

1.1 CountingCollapse.lean

The file's own header states the scope precisely; we reproduce it first because it is the most honest single paragraph in the whole artifact:

```
/-
Counting Collapse in Autoregressive Decoders
============================================

Formal core of the paper "Counting Collapse: A Formally Verified Attractor
Theory of Repetition Hallucination in Autoregressive TTS".

Setting.  An autoregressive decoder generating speech for a text prompt that
contains a phrase repeated `k` times passes through a sequence of *repetition
boundary* states `s_0, F s_0, F^2 s_0, ...`, where `F : S → S` is the composite update
the decoder applies over one full repetition of the phrase.  `S` is the decoder
state space (in practice ℝ^d with the Euclidean metric, which is a nonempty
complete metric space).

The modelling assumption is that `F` is the *same* map at every boundary.  This
is what Lemma B (`AttentionDilution.lean`) buys us: softmax attention over `k`
near-identical repeated text spans cannot tell occurrence `j` from occurrence
`j'`, so the conditioning the decoder receives at each boundary is the same up
to `O(δ + 1/k)`.

What is proved here.  *If* `F` is a contraction (`ContractingWith q F`, i.e.
`q < 1` and `F` is `q`-Lipschitz) then the number of repetitions that any
Lipschitz readout can distinguish is finite and bounded explicitly by
`log(2LC/margin) / log(1/q)`.

What is NOT proved here -- and must not be claimed.  That a real transformer
decoder satisfies the contraction hypothesis.  `q` is an empirical quantity: the
accompanying experiments estimate it as `q^` per model from the observed decay of
boundary-state distances.  The Lean artifact certifies the *implication*, not
the premise.
-/
import Mathlib

namespace SpectralTTS

open Filter Function

variable {S : Type*} [MetricSpace S] [CompleteSpace S] [Nonempty S]
variable {F : S → S} {q : NNReal}
```

Every theorem below is stated for a fixed but arbitrary nonempty complete metric space S (the decoder's hidden-state space, in practice $\mathbb{R}^d$ with the Euclidean metric) and a fixed map $F : S \to S$ (one full repetition's worth of decoder update) with contraction modulus q. These are Lean `variable` declarations, so every subsequent theorem is implicitly universally quantified over them.

orbitRadius and dist_iterate_fixedPoint_le

```
/-- The a-priori radius of the boundary-state orbit: every iterate of `F`
starting at `s` stays within `orbitRadius F s q * q ^ n` of the fixed point.
Writing it as a named quantity keeps the later bounds readable; it is the `C` of
the paper. -/
noncomputable def orbitRadius (F : S → S) (s : S) (q : NNReal) : ℝ :=
  dist s (F s) / (1 - q)

theorem orbitRadius_nonneg (hF : ContractingWith q F) (s : S) :
    0 ≤ orbitRadius F s q :=
  div_nonneg dist_nonneg hF.one_sub_K_pos.le

/-- **Theorem A(i) -- geometric convergence at repetition boundaries.**
The state after `n` repetitions is within `C q^n` of a single fixed point: the
decoder's record of how many repetitions it has produced decays geometrically.
This is `ContractingWith.apriori_dist_iterate_fixedPoint_le` restated in terms of
`orbitRadius`. -/
theorem dist_iterate_fixedPoint_le (hF : ContractingWith q F) (s : S) (n : ℕ) :
    dist (F^[n] s) (ContractingWith.fixedPoint F hF) ≤ orbitRadius F s q * (q : ℝ) ^ n := by
  have h := hF.apriori_dist_iterate_fixedPoint_le s n
  calc dist (F^[n] s) (ContractingWith.fixedPoint F hF)
      ≤ dist s (F s) * (q : ℝ) ^ n / (1 - q) := h
    _ = orbitRadius F s q * (q : ℝ) ^ n := by unfold orbitRadius; ring
```

`orbitRadius` is a bookkeeping device, not new mathematics: it is literally $C := d(s, Fs)/(1-q)$, the quantity the paper's Theorem 1 calls the orbit scale. `orbitRadius_nonneg` is immediate from $q < 1$ (so $1 - q > 0$) and $d \geq 0$.

`dist_iterate_fixedPoint_le` is Theorem A(i). It is *not* proved from scratch: mathlib already contains the Banach fixed-point theorem and, with it, an a-priori error estimate for Picard iteration, `ContractingWith.apriori_dist_iterate_fixedPoint_le`. The classical argument behind that estimate is the standard telescoping bound: since F is a q-contraction, consecutive iterates satisfy $d(F^i s, F^{i+1} s) \leq q^i d(s, Fs)$, and summing the triangle inequality over $i \geq n$ against the (unique, mathlib-constructed) fixed point s^* gives $d(F^n s, s^*) \leq \sum_{i \geq n} q^i \, d(s, Fs) = \frac{q^n}{1-q} \, d(s, Fs)$. The Lean proof here does no more than `calc`-rewrite that existing lemma through the definition of `orbitRadius`, by `ring`. The mathematical content is entirely mathlib's; the paper's contribution at this step is choosing the right restatement for what follows.

`dist_iterate_iterate_le`

```
/-- Two repetition counts are separated by at most `C (q^m + q^n)` in state
space.  Both orbits are pinned to the same fixed point, so their mutual distance
inherits the geometric decay. -/
theorem dist_iterate_iterate_le (hF : ContractingWith q F) (s : S) (m n : ℕ) :
    dist (F^[m] s) (F^[n] s) ≤ orbitRadius F s q * ((q : ℝ) ^ m + (q : ℝ) ^ n) := by
  have hm := dist_iterate_fixedPoint_le hF s m
  have hn := dist_iterate_fixedPoint_le hF s n
  calc dist (F^[m] s) (F^[n] s)
      ≤ dist (F^[m] s) (ContractingWith.fixedPoint F hF)
        + dist (ContractingWith.fixedPoint F hF) (F^[n] s) := dist_triangle _ _ _
    _ = dist (F^[m] s) (ContractingWith.fixedPoint F hF)
        + dist (F^[n] s) (ContractingWith.fixedPoint F hF) := by rw [dist_comm _ (F^[n] s)]
    _ ≤ orbitRadius F s q * (q : ℝ) ^ m + orbitRadius F s q * (q : ℝ) ^ n := by gcongr
    _ = orbitRadius F s q * ((q : ℝ) ^ m + (q : ℝ) ^ n) := by ring
```

This is the triangle inequality routed through the shared fixed point: $d(F^m s, F^n s) \leq$

$d(F^m s, s^*) + d(s^*, F^n s) \le Cq^m + Cq^n$, each half supplied by the previous theorem. `gcongr` ("generalized congruence") is a mathlib tactic that discharges a monotone combination of already-proved inequalities automatically; here it combines the two $\le$-facts term by term. This is the state-space statement behind the paper's claim that any two repetition counts land within $C(q^m + q^n)$ of each other – it does not yet say anything about a readout.

`exists_indistinguishable`

```
/-- **Theorem A(ii) -- eventual indistinguishability.**
For any tolerance `ϵ` there is a horizon `N` beyond which *all* repetition counts
produce states within `ϵ` of one another.  No amount of additional repetition
creates new state-space structure. -/
theorem exists_indistinguishable (hF : ContractingWith q F) (s : S) {ϵ : ℝ} (hϵ : 0 < ϵ) :
    ∃ N : ℕ, ∀ m n : ℕ, N ≤ m → N ≤ n → dist (F^[m] s) (F^[n] s) < ϵ := by
  set C := orbitRadius F s q with hC
  have hC0 : 0 ≤ C := orbitRadius_nonneg hF s
  have hden : 0 < 2 * C + 1 := by linarith
  obtain ⟨N, hN⟩ : ∃ N : ℕ, (q : ℝ) ^ N < ϵ / (2 * C + 1) :=
    exists_pow_lt_of_lt_one (by positivity) (by exact_mod_cast hF.1)
  refine ⟨N, fun m n hm hn => ?_⟩
  have hq0 : (0 : ℝ) ≤ (q : ℝ) := q.coe_nonneg
  have hq1 : (q : ℝ) ≤ 1 := le_of_lt (by exact_mod_cast hF.1)
  have hpm : (q : ℝ) ^ m ≤ (q : ℝ) ^ N := pow_le_pow_of_le_one hq0 hq1 hm
  have hpn : (q : ℝ) ^ n ≤ (q : ℝ) ^ N := pow_le_pow_of_le_one hq0 hq1 hn
  have hbound := dist_iterate_iterate_le hF s m n
  -- C (q^m + q^n)  ≤  2C q^N  ≤  2C ϵ/(2C+1)  <  ϵ
  have h1 : C * ((q : ℝ) ^ m + (q : ℝ) ^ n) ≤ 2 * C * (q : ℝ) ^ N := by
    have hsum : (q : ℝ) ^ m + (q : ℝ) ^ n ≤ 2 * (q : ℝ) ^ N := by linarith
    calc C * ((q : ℝ) ^ m + (q : ℝ) ^ n)
        ≤ C * (2 * (q : ℝ) ^ N) := mul_le_mul_of_nonneg_left hsum hC0
      _ = 2 * C * (q : ℝ) ^ N := by ring
  have h2 : 2 * C * (q : ℝ) ^ N ≤ 2 * C * (ϵ / (2 * C + 1)) :=
    mul_le_mul_of_nonneg_left hN.le (by linarith)
  have h3 : 2 * C * (ϵ / (2 * C + 1)) < ϵ := by
    have ht : (0 : ℝ) < ϵ / (2 * C + 1) := div_pos hϵ hden
    have heq : ϵ / (2 * C + 1) * (2 * C + 1) = ϵ := div_mul_cancel0 ϵ hden.ne'
    nlinarith [ht, heq]
  linarith [hbound, h1, h2, h3]
```

This is Theorem A(ii). Read operationally, it says: fix any tolerance $\epsilon > 0$; then there is a step count N (a horizon) past which *every pair* of repetition counts $m, n \ge N$ produces states within ϵ of each other. It is the formal counterpart of "the states the decoder visits stop being mutually distinguishable," the empirical claim tested by the capacity-gain measurement in the main paper's Section 4.3.

The proof strategy: because $q < 1$, $q^N \to 0$, so mathlib's `exists_pow_lt_of_lt_one` supplies an N with q^N below any target threshold; we choose the threshold $\epsilon/(2C+1)$ (the "+1" is there purely to keep the denominator positive even when $C = 0$, i.e. when s already *is* the fixed point). For $m, n \ge N$, monotonicity of $q^{(\cdot)}$ on $[0, 1]$ gives $q^m, q^n \le q^N$, so by `dist_iterate_iterate_le`, $d(F^m s, F^n s) \le C(q^m + q^n) \le 2Cq^N$. The chain $2Cq^N \le 2C \cdot \frac{\epsilon}{2C+1} < \epsilon$ (the last step because $\frac{2C}{2C+1} < 1$) closes the proof by `linarith`/`nlinarith` on the accumulated facts. Nothing here is specific to attention or to speech; it is a direct corollary of contraction.

`readout_gap_le`

```
/-- **Theorem A(iii) -- no Lipschitz readout can count.**
Any `L`-Lipschitz map `g : S → ℝ` -- the model's own stop-token logit, a duration
predictor, or any linear probe -- sees repetition counts `m` and `n` collapsed to
within `L C (q^m + q^n)`. -/
theorem readout_gap_le (hF : ContractingWith q F) {L : NNReal} {g : S → ℝ}
    (hg : LipschitzWith L g) (s : S) (m n : ℕ) :
    |g (F^[m] s) - g (F^[n] s)| ≤ (L : ℝ) * (orbitRadius F s q * ((q : ℝ) ^ m + (q : ℝ) ^ n)) :=
     by
  have h1 : |g (F^[m] s) - g (F^[n] s)| = dist (g (F^[m] s)) (g (F^[n] s)) :=
    (Real.dist_eq _ _).symm
  rw [h1]
  calc dist (g (F^[m] s)) (g (F^[n] s))
      ≤ (L : ℝ) * dist (F^[m] s) (F^[n] s) := hg.dist_le_mul _ _
    _ ≤ (L : ℝ) * (orbitRadius F s q * ((q : ℝ) ^ m + (q : ℝ) ^ n)) := by
        exact mul_le_mul_of_nonneg_left (dist_iterate_iterate_le hF s m n) L.coe_nonneg
```

This is the step that turns a statement about the *state space* into a statement about *anything a downstream mechanism can compute from the state.* g is an arbitrary L-Lipschitz map from S to $\mathbb{R}$: it could be the model's own stop-token logit, a linear counting probe trained after the fact, a duration predictor, or any other bounded-sensitivity read-out. The proof is two lines: rewrite $|g(a) - g(b)|$ as $\mathrm{dist}(g(a), g(b))$ on $\mathbb{R}$ (`Real.dist_eq`), bound it by $L \cdot d(a, b)$ (the definition of Lipschitz), then substitute the state-space bound already proved. No property of g beyond its Lipschitz constant is used, which is exactly why the theorem applies uniformly to greedy decoding, sampling, and beam search alike – the readout argument never inspects how a token was chosen, only how sensitive a fixed function of the state can be.

`counting_margin_le`

```
/-- **Theorem A(iv) -- the counting horizon is finite (algebraic form).**
If a readout still resolves two repetition counts `m ≤ n` with decision margin
`margin`, then `q ^ m` cannot have decayed below `margin / (2 L C)`.  Since
`q < 1`, this bounds `m`: only finitely many repetition counts admit a decision
margin at all. -/
theorem counting_margin_le (hF : ContractingWith q F) {L : NNReal} {g : S → ℝ}
    (hg : LipschitzWith L g) (s : S) {margin : ℝ} {m n : ℕ} (hmn : m ≤ n)
    (hsep : margin ≤ |g (F^[m] s) - g (F^[n] s)|) :
    margin ≤ 2 * (L : ℝ) * orbitRadius F s q * (q : ℝ) ^ m := by
  have hq0 : (0 : ℝ) ≤ (q : ℝ) := q.coe_nonneg
  have hq1 : (q : ℝ) ≤ 1 := le_of_lt (by exact_mod_cast hF.1)
  have hpn : (q : ℝ) ^ n ≤ (q : ℝ) ^ m := pow_le_pow_of_le_one hq0 hq1 hmn
  have hgap := readout_gap_le hF hg s m n
  have hC0 : 0 ≤ orbitRadius F s q := orbitRadius_nonneg hF s
  have hstep : (L : ℝ) * (orbitRadius F s q * ((q : ℝ) ^ m + (q : ℝ) ^ n))
      ≤ 2 * (L : ℝ) * orbitRadius F s q * (q : ℝ) ^ m := by
    have hsum : (q : ℝ) ^ m + (q : ℝ) ^ n ≤ 2 * (q : ℝ) ^ m := by linarith
    have hinner : orbitRadius F s q * ((q : ℝ) ^ m + (q : ℝ) ^ n)
        ≤ orbitRadius F s q * (2 * (q : ℝ) ^ m) := mul_le_mul_of_nonneg_left hsum hC0
    calc (L : ℝ) * (orbitRadius F s q * ((q : ℝ) ^ m + (q : ℝ) ^ n))
        ≤ (L : ℝ) * (orbitRadius F s q * (2 * (q : ℝ) ^ m)) :=
          mul_le_mul_of_nonneg_left hinner L.coe_nonneg
      _ = 2 * (L : ℝ) * orbitRadius F s q * (q : ℝ) ^ m := by ring
  linarith
```

This is the contrapositive step written out algebraically, before taking logarithms. Assume the readout *does* separate counts $m \le n$ by at least `margin`. Since $m \le n$ and $0 \le q \le 1$,

$q^n \le q^m$, so $q^m + q^n \le 2q^m$; feeding this into `readout_gap_le`'s bound $LC(q^m + q^n)$ gives the tighter bound $2LCq^m$. Chaining `margin` $\le LC(q^m+q^n) \le 2LCq^m$ is exactly the statement proved. In words: whatever separation a Lipschitz readout manages to extract between counts m and n, that separation is itself capped by a quantity that decays geometrically in the *smaller* of the two counts alone – which is the algebraic seed of the horizon bound.

`le_log_div_log_of_le_pow`

```
/-- Real-arithmetic bridge: a geometric quantity bounded below is bounded in its
exponent.  Used to turn `counting_margin_le` into an explicit horizon. -/
theorem le_log_div_log_of_le_pow {r c : ℝ} (hr0 : 0 < r) (hr1 : r < 1) (hc : 0 < c)
    {m : ℕ} (h : c ≤ r ^ m) : (m : ℝ) ≤ Real.log c / Real.log r := by
  have hlogr : Real.log r < 0 := Real.log_neg hr0 hr1
  have hlog : Real.log c ≤ Real.log (r ^ m) := Real.log_le_log hc h
  rw [Real.log_pow] at hlog
  rw [le_div_iff_of_neg hlogr]
  linarith [hlog]
```

A general-purpose real-analysis lemma with no reference to decoders, states, or contractions: if $0 < r < 1$, $0 < c$, and $c \le r^m$, then $m \le \log c / \log r$. The subtlety, and the reason this needs its own lemma rather than a one-line `gcongr`, is the sign flip: because $0 < r < 1$, $\log r < 0$ (`Real.log_neg`), so dividing the monotone consequence $\log c \le m \log r$ (from `Real.log_le_log` applied to $c \le r^m$, then `Real.log_pow` to expand $\log(r^m) = m \log r$) by the *negative* quantity $\log r$ reverses the inequality's direction – handled here by mathlib's `le_div_iff_of_neg`, which is precisely the division lemma for a negative denominator. This lemma is the generic engine that turns `counting_margin_le`'s algebraic inequality into an explicit bound on m in the next theorem; note that `scripts/check_lean.sh`'s axiom audit does not list this lemma by name (Section 1.3 explains why that omission is harmless).

`counting_horizon`

```
/-- **Theorem A -- Counting Collapse (explicit horizon).**
The headline statement.  Under contraction, a Lipschitz readout with decision
margin `margin > 0` can separate the repetition count `m` from any larger count
only while

    m ≤ log(margin / (2 L C)) / log q  =  log(2 L C / margin) / log(1 / q).

Beyond that horizon the decoder is provably unable to tell how many repetitions
it has already produced, so its continue/stop decision is decoupled from the
count -- the failure mode observed empirically as looping or truncation. -/
theorem counting_horizon (hF : ContractingWith q F) {L : NNReal} {g : S → ℝ}
    (hg : LipschitzWith L g) (s : S) {margin : ℝ} {m n : ℕ}
    (hq0 : (0 : ℝ) < (q : ℝ)) (hmargin : 0 < margin) (hmn : m ≤ n)
    (hsep : margin ≤ |g (F^[m] s) - g (F^[n] s)|) :
    (m : ℝ) ≤ Real.log (margin / (2 * (L : ℝ) * orbitRadius F s q)) / Real.log (q : ℝ) := by
  have hq1 : (q : ℝ) < 1 := by exact_mod_cast hF.1
  have hbound := counting_margin_le hF hg s hmn hsep
  have hC0 : 0 ≤ orbitRadius F s q := orbitRadius_nonneg hF s
  -- a nonzero decision margin forces the prefactor to be strictly positive
  have hprefix : (0 : ℝ) ≤ 2 * (L : ℝ) * orbitRadius F s q :=
    mul_nonneg (mul_nonneg (by norm_num) L.coe_nonneg) hC0
  have hpos : 0 < 2 * (L : ℝ) * orbitRadius F s q := by
    rcases lt_or_eq_of_le hprefix with h | h
```

```
    · exact h
    · exfalso
      rw [← h, zero_mul] at hbound
      linarith
  have hratio : margin / (2 * (L : ℝ) * orbitRadius F s q) ≤ (q : ℝ) ^ m := by
    rw [div_le_iff0 hpos]
    calc margin ≤ 2 * (L : ℝ) * orbitRadius F s q * (q : ℝ) ^ m := hbound
      _ = (q : ℝ) ^ m * (2 * (L : ℝ) * orbitRadius F s q) := by ring
  exact le_log_div_log_of_le_pow hq0 hq1 (by positivity) hratio
```

This is Theorem A(iv), the operative statement of the paper, combining `counting_margin_le` with the log bridge just discussed. Two details are worth pointing out because they are exactly the kind of thing a pen-and-paper proof would gloss over and Lean forces to be made explicit.

First, before the log bridge can apply, the prefactor $2LC$ must be shown *strictly* positive (the bridge lemma requires $0 < c$ for $c = \text{margin}/(2LC)$ to be well-formed as a division and for `Real.log` to behave). The proof handles the boundary case directly: if $2LC$ were 0, then `hbound` (margin $\leq 2LC \cdot q^m$) would force margin ≤ 0, contradicting the hypothesis margin > 0 – so the very existence of a positive decision margin is what rules out the degenerate case $L = 0$ or $C = 0$ (a zero-Lipschitz-constant readout, or an already-converged orbit). This is the `rcases lt_or_eq_of_le` case split ending in `exfalso`.

Second, the theorem as stated in Lean gives $m \leq \log\big(\text{margin}/(2LC)\big)/\log q$, while the main paper's Equation (1) writes the horizon as $\log(2LC/\text{margin})/\log(1/q)$. These are the same quantity: since $\text{margin}/(2LC) \in (0,1)$ whenever the readout achieves a real separation (so its log is negative) and $q \in (0,1)$ (so $\log q$ is also negative), $\log(a)/\log(b) = \log(1/a)/\log(1/b)$ for $a = \text{margin}/(2LC)$, $b = q$ – a negative-over-negative flips to a positive-over-positive with the reciprocal arguments. The Lean statement and the paper's Equation (1) are the identical horizon, just written with the sign convention that falls out of `le_log_div_log_of_le_pow` versus the convention chosen for exposition.

1.2 `AttentionDilution.lean`

```
/-
Attention Dilution
==================

Lemma B of "Counting Collapse: A Formally Verified Attractor Theory of
Repetition Hallucination in Autoregressive TTS".

Setting.  A decoder step attends over the `k` text positions occupied by `k`
repetitions of the same phrase.  Because those positions carry (near-)identical
content, their attention logits are near-identical too: we assume only that the
logits lie within a spread `δ` of one another, which is a directly measurable
quantity.

What is proved.  Every occurrence receives attention weight in
`[e^{-δ}/k, e^{δ}/k]`; consequently the attention profile over the repeated
block is within `(e^{δ} - e^{-δ})/k` of uniform, and its entropy is at least
`log k - δ`.  As `k` grows the profile flattens: the decoder cannot use
attention weight to identify *which* occurrence it is currently rendering.

Why it matters.  This licenses the modelling assumption of
`CountingCollapse.lean` -- that the conditioning at every repetition boundary is
the same map `F`.  The link is quantitative but informal (an `O(δ + 1/k)`
perturbation of the conditioning yields an `O(δ + 1/k)` perturbation of `F`); the
```

```
paper states it as a remark, not as a formalized theorem.
-/
import Mathlib

namespace SpectralTTS

open Finset Real

variable {k : ℕ}

/-- Softmax over the `k` attention logits of a repeated text block. -/
noncomputable def softmax (z : Fin k → ℝ) (i : Fin k) : ℝ :=
  Real.exp (z i) / ∑ j, Real.exp (z j)

theorem sum_exp_pos [NeZero k] (z : Fin k → ℝ) : 0 < ∑ j, Real.exp (z j) :=
  Finset.sum_pos (fun j _ => Real.exp_pos (z j))
    ⟨⟨0, Nat.pos_of_ne_zero (NeZero.ne k)⟩, mem_univ _⟩

theorem softmax_nonneg [NeZero k] (z : Fin k → ℝ) (i : Fin k) : 0 ≤ softmax z i :=
  div_nonneg (Real.exp_pos _).le (sum_exp_pos z).le
```

`softmax` is defined exactly as one would expect – $\mathrm{softmax}(z)_i = e^{z_i}/\sum_j e^{z_j}$ over the k logits of a repeated block – and `sum_exp_pos`/`softmax_nonneg` are routine positivity facts needed to make the subsequent division lemmas well-formed (the denominator is a finite sum of strictly positive terms over a nonempty index type, guaranteed by the `[NeZero k]` instance).

`sum_softmax`

```
/-- The softmax profile is a probability distribution. -/
theorem sum_softmax [NeZero k] (z : Fin k → ℝ) : ∑ i, softmax z i = 1 := by
  unfold softmax
  rw [← Finset.sum_div, div_self (sum_exp_pos z).ne']
```

Establishes that `softmax` really is a probability distribution over the k occurrences: $\sum_i e^{z_i}/\sum_j e^{z_j} = \left(\sum_i e^{z_i}\right)/\sum_j e^{z_j} = 1$, by pulling the common denominator out of the sum (`Finset.sum_div`, used backwards) and cancelling it against itself. This is a prerequisite for the entropy statement later, since Shannon entropy is only meaningful over a genuine distribution.

`softmax_le` and `le_softmax`

```
/-- **Lemma B (upper bound).**  If all logits in the repeated block lie within
`δ` of one another, no single occurrence can capture more than `e^{δ}/k` of the
attention mass. -/
theorem softmax_le [NeZero k] (z : Fin k → ℝ) {δ : ℝ} (hz : ∀ a b, z a - z b ≤ δ) (i : Fin k) :
    softmax z i ≤ Real.exp δ / k := by
  have hk : (0 : ℝ) < k := Nat.cast_pos.mpr (Nat.pos_of_ne_zero (NeZero.ne k))
  -- every term of the denominator is at least `exp (z i - δ)`
  have hlow : (k : ℝ) * Real.exp (z i - δ) ≤ ∑ j, Real.exp (z j) := by
    have hterm : ∀ j ∈ (univ : Finset (Fin k)), Real.exp (z i - δ) ≤ Real.exp (z j) := by
      intro j _
      exact Real.exp_le_exp.mpr (by linarith [hz i j])
    calc (k : ℝ) * Real.exp (z i - δ)
        = ∑ _j : Fin k, Real.exp (z i - δ) := by
          rw [Finset.sum_const, card_univ, Fintype.card_fin, nsmul_eq_mul]
      _ ≤ ∑ j, Real.exp (z j) := Finset.sum_le_sum hterm
```

```
  unfold softmax
  rw [div_le_div_iff0 (sum_exp_pos z) hk]
  have hkey : Real.exp (z i) * (k : ℝ) = Real.exp δ * ((k : ℝ) * Real.exp (z i - δ)) := by
    rw [Real.exp_sub]
    field_simp
  rw [hkey]
  exact mul_le_mul_of_nonneg_left hlow (Real.exp_pos δ).le

/-- **Lemma B (lower bound).**  Symmetrically, no occurrence can be starved
below `e^{-δ}/k`. -/
theorem le_softmax [NeZero k] (z : Fin k → ℝ) {δ : ℝ} (hz : ∀ a b, z a - z b ≤ δ) (i : Fin k) :
    Real.exp (-δ) / k ≤ softmax z i := by
  have hk : (0 : ℝ) < k := Nat.cast_pos.mpr (Nat.pos_of_ne_zero (NeZero.ne k))
  have hhigh : ∑ j, Real.exp (z j) ≤ (k : ℝ) * Real.exp (z i + δ) := by
    have hterm : ∀ j ∈ (univ : Finset (Fin k)), Real.exp (z j) ≤ Real.exp (z i + δ) := by
      intro j _
      exact Real.exp_le_exp.mpr (by linarith [hz j i])
    calc ∑ j, Real.exp (z j)
        ≤ ∑ _j : Fin k, Real.exp (z i + δ) := Finset.sum_le_sum hterm
      _ = (k : ℝ) * Real.exp (z i + δ) := by
          rw [Finset.sum_const, card_univ, Fintype.card_fin, nsmul_eq_mul]
  unfold softmax
  rw [div_le_div_iff0 hk (sum_exp_pos z)]
  have hkey : Real.exp (-δ) * ((k : ℝ) * Real.exp (z i + δ)) = Real.exp (z i) * (k : ℝ) := by
    rw [Real.exp_add, Real.exp_neg]
    field_simp
  calc Real.exp (-δ) * ∑ j, Real.exp (z j)
      ≤ Real.exp (-δ) * ((k : ℝ) * Real.exp (z i + δ)) :=
        mul_le_mul_of_nonneg_left hhigh (Real.exp_pos _).le
    _ = Real.exp (z i) * (k : ℝ) := hkey
```

These two theorems together are Lemma B's headline bound. The hypothesis `hz` says the k logits of the repeated block are mutually within δ of each other – a directly measurable spread, not an assumption about the mechanism producing the logits.

For the upper bound: since $z_i - z_j \le \delta$ for every j, each term of the denominator satisfies $e^{z_j} \ge e^{z_i-\delta}$, so summing k copies of that lower bound gives $\sum_j e^{z_j} \ge k\, e^{z_i-\delta}$. Substituting into the definition of softmax, $\mathrm{softmax}(z)_i = e^{z_i}/\sum_j e^{z_j} \le e^{z_i}/(k\, e^{z_i-\delta}) = e^{\delta}/k$. The lower bound is the mirror argument with the roles of i and j swapped in the hypothesis ($z_j - z_i \le \delta$, i.e. $e^{z_j} \le e^{z_i+\delta}$ for every j), giving $\sum_j e^{z_j} \le k\, e^{z_i+\delta}$ and hence $\mathrm{softmax}(z)_i \ge e^{-\delta}/k$. Both proofs are the same three-step pattern – bound every summand, sum k identical bounds via `Finset.sum_const`, divide – executed twice with the inequality direction and the sign of δ flipped.

softmax_dist_le

```
/-- **Occurrence indistinguishability.**  The attention profile over a block of
`k` repeated spans is within `(e^{δ} - e^{-δ})/k` of uniform.  For fixed `δ`
this vanishes as `1/k`: attention weight carries no information about *which*
repetition is being attended to. -/
theorem softmax_dist_le [NeZero k] (z : Fin k → ℝ) {δ : ℝ} (hz : ∀ a b, z a - z b ≤ δ)
    (i j : Fin k) :
    |softmax z i - softmax z j| ≤ (Real.exp δ - Real.exp (-δ)) / k := by
  have hsplit : (Real.exp δ - Real.exp (-δ)) / (k : ℝ)
      = Real.exp δ / (k : ℝ) - Real.exp (-δ) / (k : ℝ) := by ring
  have hi_up := softmax_le z hz i
  have hj_up := softmax_le z hz j
```

```
  have hi_lo := le_softmax z hz i
  have hj_lo := le_softmax z hz j
  rw [abs_sub_le_iff]
  constructor <;> linarith
```

A direct corollary: both $\mathrm{softmax}(z)_i$ and $\mathrm{softmax}(z)_j$ lie in the interval $[e^{-\delta}/k,\, e^{\delta}/k]$, an interval of width $(e^{\delta} - e^{-\delta})/k$, so their difference cannot exceed that width in either direction (`abs_sub_le_iff` splits $|x-y| \le c$ into the two one-sided inequalities $x - y \le c$ and $y - x \le c$, both closed by `linarith` from the four bounds already in scope). For fixed δ this bound is $\Theta(1/k)$: as the block grows, no two occurrences' attention weights can differ by more than an amount vanishing in k – the formal content of "attention weight carries vanishing information about which occurrence is being rendered."

entropy_ge

```
/-- **Attention entropy grows like `log k`.**  With `p i = softmax z i`, the
Shannon entropy of the profile over the repeated block satisfies
`H(p) ≥ log k - δ`: near-uniform attention over `k` identical spans is maximally
uninformative, so the decoder's effective conditioning becomes the block mean and
loses occurrence identity. -/
theorem entropy_ge [NeZero k] (z : Fin k → ℝ) {δ : ℝ} (hz : ∀ a b, z a - z b ≤ δ) :
    Real.log k - δ ≤ ∑ i, softmax z i * (-Real.log (softmax z i)) := by
  have hk : (0 : ℝ) < k := Nat.cast_pos.mpr (Nat.pos_of_ne_zero (NeZero.ne k))
  have key : ∀ i : Fin k, (Real.log k - δ) * softmax z i
      ≤ softmax z i * (-Real.log (softmax z i)) := by
    intro i
    have hup := softmax_le z hz i
    have hlo : 0 < softmax z i := by
      have h1 : Real.exp (-δ) / (k : ℝ) ≤ softmax z i := le_softmax z hz i
      have h2 : (0 : ℝ) < Real.exp (-δ) / (k : ℝ) := by positivity
      linarith
    have hlog : Real.log (softmax z i) ≤ δ - Real.log k := by
      have h1 : Real.log (softmax z i) ≤ Real.log (Real.exp δ / k) := Real.log_le_log hlo hup
      rwa [Real.log_div (Real.exp_pos δ).ne' hk.ne', Real.log_exp] at h1
    have hneg : Real.log k - δ ≤ -Real.log (softmax z i) := by linarith
    calc (Real.log k - δ) * softmax z i
        ≤ (-Real.log (softmax z i)) * softmax z i :=
          mul_le_mul_of_nonneg_right hneg hlo.le
      _ = softmax z i * (-Real.log (softmax z i)) := mul_comm _ _
  calc Real.log k - δ
      = (Real.log k - δ) * ∑ i, softmax z i := by rw [sum_softmax z]; ring
    _ = ∑ i, (Real.log k - δ) * softmax z i := by rw [Finset.mul_sum]
    _ ≤ ∑ i, softmax z i * (-Real.log (softmax z i)) := Finset.sum_le_sum (fun i _ => key i)
```

The entropy floor. Write $H(p) = \sum_i p_i(-\log p_i)$ for the Shannon entropy of the attention profile $p_i = \mathrm{softmax}(z)_i$. The proof first shows a pointwise inequality (`key`): for every occurrence i, $(\log k - \delta)\, p_i \le p_i(-\log p_i)$. This follows because `softmax_le` gives $p_i \le e^{\delta}/k$, hence (taking logs, valid since $p_i > 0$ by `le_softmax`) $\log p_i \le \delta - \log k$, i.e. $-\log p_i \ge \log k - \delta$; multiplying both sides of that by the nonnegative p_i preserves the inequality. Summing the pointwise bound over all k occurrences and using $\sum_i p_i = 1$ (`sum_softmax`) turns $\sum_i (\log k - \delta)\, p_i$ into exactly $\log k - \delta$, giving $H(p) \ge \log k - \delta$. As $k \to \infty$ for fixed δ, the minimum possible entropy of the attention profile over the repeated block grows without bound like $\log k$: attention over a large repeated block cannot be concentrated, no matter how the model's underlying logits are computed, subject only to their spread being bounded by δ.

1.3 The axiom audit

`scripts/check_lean.sh` is the verification gate. It performs three checks in sequence: (1) `lake build` succeeds, i.e. the project type-checks against the pinned mathlib revision; (2) `grep -rn "sorry"` over `SpectralTTS/` and `SpectralTTS.lean` finds no occurrence, i.e. no proof obligation anywhere in the source was left unproved with the `sorry` placeholder; and (3) an axiom audit, reproduced verbatim below, that prints the transitive axiom dependency of eleven exported theorems and fails the build if the string `sorryAx` appears anywhere in that output.

```
== axiom audit
cat > /tmp/spectraltts_axioms.lean <<'EOF'
import SpectralTTS
open SpectralTTS
#print axioms SpectralTTS.dist_iterate_fixedPoint_le
#print axioms SpectralTTS.dist_iterate_iterate_le
#print axioms SpectralTTS.exists_indistinguishable
#print axioms SpectralTTS.readout_gap_le
#print axioms SpectralTTS.counting_margin_le
#print axioms SpectralTTS.counting_horizon
#print axioms SpectralTTS.softmax_le
#print axioms SpectralTTS.le_softmax
#print axioms SpectralTTS.softmax_dist_le
#print axioms SpectralTTS.entropy_ge
#print axioms SpectralTTS.sum_softmax
EOF
OUT=$(lake env lean /tmp/spectraltts_axioms.lean 2>&1)
echo "$OUT"
if echo "$OUT" | grep -q "sorryAx"; then
  echo "FAIL: sorryAx in dependency closure"; exit 1
fi
```

What the three admitted axioms mean. Lean's kernel tracks, for every proof term, exactly which axioms it ultimately rests on beyond pure type-theoretic reduction. Three axioms are considered standard and non-suspicious throughout ordinary (i.e. classical) formalised mathematics, and mathlib is built expecting all three:

- **`propext`** (propositional extensionality): two propositions that are provably logically equivalent are equal as terms of type `Prop`. This licenses rewriting one true statement for another and is used pervasively any time an equality between propositions, rather than merely an if-and-only-if, is needed internally.

- **`Classical.choice`**: the axiom of choice, in the form that every nonempty type has a term. Mathlib is a classical (not constructive) library and uses this axiom routinely – for instance, to decide the truth of an arbitrary proposition about real numbers (`Real.log`'s case splits, order comparisons, and the existence statement in `ContractingWith.fixedPoint` all rest on classical reasoning of this kind).

- **`Quot.sound`**: quotient soundness. Lean 4 builds quotient types (used, transitively, to construct $\mathbb{R}$ as a quotient of Cauchy sequences, `NNReal` as a subtype of that, and `Finset` as a quotient of `Multiset` which is itself a quotient of `List`) and this axiom is what makes the definitional behaviour of quotients sound at the kernel level.

Essentially every nontrivial mathlib theorem depends on some subset of these three; a formal result is treated as unconditionally established in this ecosystem exactly when its axiom dependency is a subset of {`propext`, `Classical.choice`, `Quot.sound`}.

What `sorryAx` would mean. Whenever a proof anywhere in a term's transitive dependency closure contains an unproved hole – written `sorry`, or produced by an incomplete tactic block – Lean's elaborator inserts a synthetic axiom called `sorryAx` standing in for "trust me." A theorem whose dependency closure contains `sorryAx` is not actually proved: it is, formally, an assumption. Because `sorryAx` propagates through every downstream use of the offending lemma, a single unproved hole anywhere upstream – even inside a helper three calls deep, even hypothetically inside a buggy mathlib lemma – would surface in the audit of every theorem that transitively depends on it. This is why the `grep -rn "sorry"` textual scan (step 2) and the semantic `#print axioms` scan (step 3) are both run: the grep catches `sorry` written directly in this project's own two files, while the axiom audit is the only check that would also catch a hole hidden inside a project-local tactic-generated term that the textual scan cannot see, or (in principle) an inconsistency imported from mathlib itself. Passing both certifies that all eleven audited theorems reduce, through Lean's kernel type-checker, to a proof term whose only axioms are the three standard ones above.

Coverage note. The audit list contains eleven theorems: the seven from `CountingCollapse.lean` covered in Section 1.1 except `le_log_div_log_of_le_pow`, plus the four listed and named theorems from `AttentionDilution.lean` (`softmax_le`, `le_softmax`, `softmax_dist_le`, `entropy_ge`) and `sum_softmax`. `le_log_div_log_of_le_pow` is not itself named in the `#print axioms` list, but this is not a coverage gap: it is a direct dependency of `counting_horizon`, so Lean's kernel necessarily re-checks it (and everything *it* depends on) while type-checking `counting_horizon`, and any axiom it introduced would appear in `counting_horizon`'s own reported axiom set. Auditing `counting_horizon` therefore already certifies `le_log_div_log_of_le_pow` transitively. The same reasoning covers the unlisted helper lemmas `orbitRadius_nonneg`, `sum_exp_pos`, and `softmax_nonneg`: each is a dependency of an audited theorem.

What is and is not proved (restated plainly). The Lean artifact certifies an implication, not a fact about trained transformers. Concretely: *if* a decoder's per-repetition update map F is a genuine q-contraction ($q < 1$, Lipschitz) on its hidden-state space, *then* Theorem A(i)–(iv) follow with no further assumptions, machine-checked down to three standard axioms. Whether any given trained decoder's F actually is a contraction is not something proved here or provable by this kind of argument at all – it is an empirical property of a specific trained network, measured per model as $\hat{q}$ in the main paper's Section 4 and Section 4 below. Symmetrically, Lemma B (attention dilution) is unconditional – it holds for any bounded-spread logits, which is a directly measurable, not assumed, property of a forward pass – but its role is only to license the informal step from "attention is diluted" to "the per-repetition map is approximately the same map at every boundary" (Assumption 1 of the main paper), and that step is itself not formalised; see Section 2.

2 What is deliberately not formalized

The main paper's Assumption 1 (periodic conditioning: there is a single map F with $s_{m+1} = F(s_m)$ for every boundary m) is presented as "what Lemma B buys us." This is true informally and is stated as such in both the paper and the `AttentionDilution.lean` header: *"The link is quantitative but*

informal (an $O(\delta + 1/k)$ perturbation of the conditioning yields an $O(\delta + 1/k)$ perturbation of F); the paper states it as a remark, not as a formalized theorem." This section says precisely what that remark asserts, why it was not formalised, and what a full formal treatment would need to add. Nothing here is proved in Lean; this section is pen and paper by design, and we are explicit about that so the boundary between the machine-checked and the informal parts of the argument is never ambiguous to a reader.

2.1 What the remark actually claims

Lemma B, as formalised, is a statement about a single decoder step's attention profile over the k text spans occupied by k repetitions: if those spans' logits lie within spread δ, the resulting softmax weights are within $(e^{\delta} - e^{-\delta})/k$ of uniform (`softmax_dist_le`) and the profile's entropy is at least $\log k - \delta$ (`entropy_ge`). Neither of these is yet a statement about F, the map *between repetition boundaries* that Theorem A's contraction hypothesis is about. The step from one to the other requires two things Lemma B does not supply:

1. A model of *what the decoder update depends on.* F, as used in `CountingCollapse.lean`, is an autonomous map $S \to S$ with no explicit dependence on anything besides the current state s. In reality the per-repetition update is a function of the current state *and* the conditioning context the decoder attends to at that step – write this as $F_c : S \to S$ for a context c drawn from some space of possible conditioning contexts (e.g. the vector of attended text values, not merely the attention weights Lemma B bounds). Autonomy of F is only recovered *if* the sequence of contexts $c_0, c_1, \ldots$ actually visited at successive repetition boundaries is (nearly) constant.

2. A translation of Lemma B's bound on *attention weights* into a bound on the resulting *context vector*. Lemma B bounds $|\mathrm{softmax}(z)_i - \mathrm{softmax}(z)_j|$, a statement purely about the weights. The context the decoder actually receives is a weight-averaged combination of *value* vectors, and Lemma B says nothing about those values – it only constrains how uniformly the weights spread across positions whose values are assumed, but not shown, to be similar because the k repeated spans carry near-identical text.

Granting both informally – flat attention over near-identical spans yields a near-constant context vector, to an error the paper writes as $O(\delta + 1/k)$ – the remark's claim is: a family of maps $\{F_c\}$ that is uniformly Lipschitz in c (i.e. nearby contexts produce nearby updates, $d(F_c(s), F_{c'}(s)) \leq \kappa\, d_C(c, c')$ for some constant κ and every s), fed a sequence of contexts $c_0, c_1, \ldots$ that are all within $O(\delta + 1/k)$ of some reference context c^*, behaves *approximately* like the single autonomous map F_{c^*}, with an error that itself scales as $O(\delta + 1/k)$ per step. This is the sense in which Assumption 1 is "bought" by Lemma B: not literally, but up to a perturbation that shrinks as the repeated block grows.

2.2 Why this was not formalised

Formalising the remark above is a strictly larger undertaking than formalising Theorem A and Lemma B as stated, for reasons that are worth being concrete about rather than hand-waving as "future work":

1. **A new object.** The current development has no type for "conditioning context" or for a context-indexed family of maps $\{F_c\}_{c \in C}$; this would need to be introduced, along with a

metric d_C on contexts and a joint-Lipschitz hypothesis relating d_C to the induced variation in F_c. None of this exists in either Lean file today.

2. **A quantitative re-derivation in the context metric, not the weight metric.** `softmax_dist_le` bounds a difference of scalar attention weights. Bounding the resulting difference in *context vectors* $d_C(c_i, c_j)$ requires an additional Lipschitz-type assumption on the value vectors being attended to (that repeated spans carry not just diluted attention but near-identical *values*), which is a second, currently unstated and unmeasured premise, and then a short but genuine argument (a weighted-average version of the triangle inequality) to convert a bound on weight differences plus a bound on value differences into a bound on the resulting weighted sum.

3. **A perturbed-contraction (shadowing) lemma.** Even granting (1) and (2), Theorem A's proof chain (Section 1) is stated for a single autonomous F. Propagating a per-step drift of size $\rho = O(\delta + 1/k)$ through it requires a genuinely different fixed-point argument: something in the spirit of "a sequence of q-contractions $F_{c_0}, F_{c_1}, \ldots$ whose contexts all lie within ρ of a common reference point produces an orbit that shadows the autonomous orbit of F_{c^*} up to an additive error that itself stabilises geometrically, to a limiting value on the order of $\kappa\rho/(1-q)$." This is a standard *shape* of result in the perturbation theory of dynamical systems (a discrete analogue of structural stability / shadowing for contractions), but it is not a lemma mathlib currently provides in a directly reusable form for this setting, and instantiating or proving it was out of scope for this artifact.

4. **Propagating the correction through the horizon.** Granting (3), every inequality in Theorem A(i)–(iv) would pick up an additive term of order $\kappa\rho/(1-q)$ on top of the geometric Cq^m decay, and the horizon of Equation (1) would gain a k-dependent correction. Because $\rho = O(\delta + 1/k)$ shrinks precisely as k (and hence m, the quantity the horizon bounds) grows, the correction is plausibly a lower-order effect in the regime the paper cares about – but showing this rigorously is a two-scale argument (the perturbation shrinking in the same variable the main bound is stated over) that requires genuinely new mathematical content, not routine bookkeeping, and was not attempted.

2.3 What a full treatment would need

Concretely, extending the formal artifact to close this gap would require, in order: (a) a formal definition of a conditioning-context space C with a metric d_C and a context-indexed family $F : C \to S \to S$; (b) a hypothesis package bundling (i) κ-Lipschitz continuity of F in its context argument, uniformly in the state argument, and (ii) a quantitative bound, in d_C, on the distance between the contexts realised at successive repetition boundaries – the latter to be derived from a strengthened version of Lemma B that bounds the attended *value* vectors' contribution, not only the attention weights; (c) a perturbed-orbit / shadowing lemma for metric-space contractions under a bounded per-step context drift, proved or sourced from mathlib; and (d) a re-derivation of Theorem A(i)–(iv) with the additive $O(\kappa\rho/(1-q))$ correction carried through explicitly, together with an argument (formal or at least made quantitatively precise) that this correction is dominated by the main geometric term in the k-range where the horizon is informative. None of (a)–(d) is present in the current artifact; the current Lean development proves Theorem A and Lemma B exactly as stated in Section 1, and the bridge between them remains, honestly, a remark.

3 The benchmark in full

The benchmark is built by `data/stimuli/make_stimuli.py` and written to `data/stimuli/stimuli.jsonl`, one JSON object per line. It contains 180 items across five families, confirmed by direct count against the generated file:

Table 1: Stimulus families.

Family	Count	What it tests
`word_rep`	60	a single word repeated k times in a fixed carrier
`control_word`	54	the same carrier, k *distinct* filler words (no repetition)
`sentence_rep`	32	a whole short sentence repeated k times
`twisters`	24	a tongue-twister sentence (with internal repetition) repeated k times
`numbers`	10	English number phrases with intrinsic digit-word repetition
Total	180	

Every item carries an explicit `item_id`, `family`, `template`, `text`, `target_unit`, `k`, and `expected_count` – a ground-truth number of occurrences of the target unit, so a generation can be scored by counting rather than by string match against a reference transcript. `control_word` items additionally carry `control_of` (the `item_id` of the repeated item they are matched to) and `control_units` (the k distinct fillers actually substituted).

3.1 The k-ladder

`word_rep` and `control_word` items are built at

$$k \in \{1, 2, 3, 4, 6, 8, 12, 16, 24, 32\}$$

(`K_LADDER` in `make_stimuli.py`). The ladder is dense at the low end and log-spaced above; the source comment explains why directly: the collapse point k^* is expected in the 4–16 range, so that is where resolution is spent, without paying the instrumentation and generation cost of testing every integer k up to 32. `sentence_rep` uses a truncated ladder $k \in \{1, 2, 3, 4, 6, 8, 12, 16\}$ (`SENTENCE_K`), since a full sentence repeated 32 times would be prohibitively long to generate and transcribe. `twisters` use only $k \in \{1, 2, 4\}$, since a twister sentence already contains internal repetition (Table 5) and the family exists to probe alliterative/phonological stress rather than long-range counting. `numbers` items are not built on a k-ladder at all: each number phrase is a single fixed string with its own intrinsic repetition count (Table 3).

3.2 Word-repetition carrier templates

Table 2 reproduces all six carrier templates (`WORD_TEMPLATES`) verbatim: the fixed prefix and suffix, the target word repeated k times between them, and the pool of eight distinct fillers used to build the matched control at the same k. An item's text is `prefix + " " + (target repeated k times) + " " + suffix`; a control's text substitutes `prefix + " " + (k distinct fillers, cycled through the pool) + " " + suffix`, guaranteeing the target word itself never appears in its own control.

Table 2: Word-repetition carrier templates (`WORD_TEMPLATES`, `data/stimuli/make_stimuli.py`).

ID	Prefix	Target	Suffix	Filler pool (for the control)
t1	The dog was	very	big and it ran across the field.	quite, really, truly, fairly, rather, somewhat, extremely, notably
t2	She said	no	to the offer and then left the room.	yes, maybe, sure, fine, okay, well, right, hmm
t3	He walked	far	into the forest before turning back.	deep, fast, long, wide, high, low, near, past
t4	The light was	blue	before the storm arrived.	green, bright, pale, dim, warm, cold, sharp, soft
t5	The runner moved	quick	along the narrow path.	swift, smooth, steady, light, sharp, loose, tight, clean
t6	They were	really	tired after the long journey.	truly, quite, very, rather, deeply, clearly, plainly, surely

Why these six target words. The source comment states the selection criterion directly: targets are "chosen for ASR robustness: common, monosyllabic-or-disyllabic, no homophone confusions, and unlikely to be swallowed by Whisper's LM prior." Concretely, *very*, *no*, *far*, *blue*, *quick*, and *really* are all common, short, phonetically distinct from every filler in their own pool, and none are function words or discourse markers that an ASR language-model prior might silently elide or merge with an adjacent word – a real risk for a word repeated up to 32 times in a row, which is exactly the regime this benchmark stresses. This choice matters because the behavioural metric (Section 4) counts occurrences of the target unit in the ASR transcript; a target word prone to ASR mis-transcription independent of the TTS model's behaviour would contaminate that count with ASR noise rather than TTS behaviour.

3.3 Number phrases

Table 3 reproduces all ten number-phrase stimuli (`NUMBER_PHRASES`) verbatim, with the target digit-word and its true occurrence count. Text is capitalised and given a trailing period at generation time (`text.capitalize() + "."`). The family's rationale, from the source comment: "English number words are the natural repetition stress test: the same digit-word recurs at several magnitudes and the model must keep an internal place-value counter that repetition-attractor dynamics would destroy." Unlike `word_rep`, the repeated unit here is not adjacent tokens but a digit-word recurring at different magnitude positions (hundreds, thousands, millions), so a correct rendering additionally requires tracking place value, not merely repeating a token a fixed number of times.

At scoring time (Section 4), a numeral rendered by Whisper as digits (e.g. `"666,666"`) is algorithmically expanded back into English number words by `expand_number_token` in `src/common/score_counts.py`, so a model's output can be counted in the same normalised token space as every other family; a numeral longer than 12 digits (i.e. a looping model's runaway output, which Whisper can render as an absurdly long integer) is deliberately *not* run through place-value expansion and is instead read off digit-by-digit, on the grounds that place-value names are undefined past *billion* and a digit-wise reading is the semantically correct interpretation of what a looping decoder actually spoke.

Table 3: Number-phrase stimuli (`NUMBER_PHRASES`, `data/stimuli/make_stimuli.py`).

ID	Phrase	Target	Count
n01	six hundred sixty six thousand six hundred sixty six	six	6
n02	seven hundred seventy seven thousand seven hundred seventy seven	seven	6
n03	nine hundred ninety nine thousand nine hundred ninety nine	nine	6
n04	three hundred thirty three thousand three hundred thirty three	three	6
n05	six hundred sixty six million six hundred sixty six thousand six hundred sixty six	six	9
n06	eight hundred eighty eight thousand eight hundred eighty eight	eight	6
n07	five hundred fifty five	five	3
n08	four hundred forty four	four	3
n09	two hundred twenty two million two hundred twenty two thousand two hundred twenty two	two	9
n10	one hundred eleven thousand one hundred eleven	one	5

3.4 Sentence repetition and tongue twisters

`sentence_rep` repeats one of four short sentences $k \in \{1, 2, 3, 4, 6, 8, 12, 16\}$ times (Table 4); the scored target is a single content word within the sentence. `twisters` instead repeat a whole tongue-twister sentence $k \in \{1, 2, 4\}$ times (Table 5); because several classic twisters already repeat a word or a phoneme cluster *within one copy* of the sentence, the ground-truth count is expected_count = per-copy occurrences $\times$ k, not k itself. Two twisters (`w6`, `w7`) are alliterative rather than word-repetitive – their nominal "target" is a short substring (`"s"`, `"fr"`) that occurs in many unrelated words, so the source sets their per-copy occurrence count to 0 rather than attempting a substring count that would be semantically meaningless; these two items therefore always carry `expected_count`= 0 and exercise phonological/alliterative rendering stress rather than the counting metric.

Table 4: Sentence-repetition stimuli (`SENTENCES`, `data/stimuli/make_stimuli.py`). $k \in \{1, 2, 3, 4, 6, 8, 12, 16\}$ for every row.

ID	Sentence	Target
s1	The bell rang twice.	bell
s2	He counted the stones.	stones
s3	Rain fell on the roof.	rain
s4	The door stayed open.	door

3.5 The matched-control construction

Every `word_rep` item at $k \geq 2$ has a corresponding `control_word` item ($k = 1$ is defined to be its own control, since there is no repetition to remove): the same carrier prefix and suffix, the same word count, with the k copies of the target replaced by k *distinct* words drawn from that template's filler pool (cycled and skipped past any accidental collision with the target itself, so the target word

Table 5: Tongue-twister stimuli (`TWISTERS`, `data/stimuli/make_stimuli.py`). "Per-copy" is the number of times the target occurs within a single copy of the sentence; `expected_count` = per-copy $\times k$ for $k \in \{1, 2, 4\}$.

ID	Sentence	Target	Per-copy
w1	She sells sea shells by the sea shore.	sea	2
w2	Peter Piper picked a peck of pickled peppers.	peck	1
w3	How much wood would a woodchuck chuck.	chuck	1
w4	Red lorry yellow lorry red lorry yellow lorry.	lorry	4
w5	The sixth sick sheikh's sixth sheep is sick.	sixth	2
w6	Six slick slim sycamore saplings.	s (alliteration only)	0
w7	Fresh French fried fly fritters.	fr (alliteration only)	0
w8	Truly rural truly rural truly rural.	rural	3

is guaranteed never to appear in its own control). This is the construction underlying the paper's central dissociation test (Section 4.2 of the main paper, Figure 1(b)): the control holds carrier syntax and word count fixed while removing exactly one variable, periodicity, so that a difference in outcome between the repeated item and its control cannot be explained by sequence length, generation duration, or the amount of text the model has to render.

3.6 The `boundary_units` field

`word_rep` and `sentence_rep` items carry `boundary_units`: the ordered list of the k words (all copies of the target, for a repeated item) whose text positions delimit the k repetition boundaries. `control_word` items carry the analogous list of the k distinct fillers actually substituted (duplicated under the key `control_units` for the separate purpose of scoring: was each of the k fillers rendered at all). This field exists so that the boundary-location procedure of `src/common/boundaries.py` (`unit_columns`, Section 5) is handed an explicit, model-agnostic list of search targets rather than having to special-case repeated versus control text – both item types get their boundaries located by the identical left-to-right, non-rewinding string-search procedure, over the identical field, which is what makes the fitted decay rates of repeated and control items directly comparable and not confounded by using two different localisation methods for the two arms of the comparison. `numbers` and `twisters` items carry no `boundary_units` field at all: numbers have no single repeating word-level unit (the repetition is a digit-word recurring at different magnitude positions), and twisters repeat a whole sentence rather than a word, so neither family has a boundary structure at the granularity this field is designed for.

3.7 The instrumented subset

An item is marked `instrumented=true` (and therefore receives the teacher-forced hidden-state/attention capture pass described in Section 6) exactly when its family is one of {`word_rep`, `sentence_rep`, `control_word`} and $k \geq 2$. The source comment gives the rationale directly: "Twisters have no single repeating unit at $k = 1$, numbers are short; both are behavioural-only." $k \geq 2$ is required because at $k = 1$ there is by definition no repetition boundary to instrument. This yields 54 (`word_rep`, $k \geq 2$)+54 (`control_word`, all $k \geq 2$ by construction)+28 (`sentence_rep`, $k \geq 2$) = 136 instrumented items, matching the count written by `make_stimuli.py` at generation time and the `NInstrumented` macro emitted by `analysis/make_numbers.py`.

4 Measurement protocol in full

4.1 The conjunctive correctness criterion

The central behavioural question – given text asking for k repetitions, how many did the model actually produce – is answered by reconciling two independent, differently-biased estimators, implemented in `src/common/score_counts.py`.

Count A (transcript-based). The generated audio is transcribed with Whisper large-v3 (`src/common/asr_transcribe.py`), the transcript is normalised (lower-cased, punctuation stripped except commas needed for numeral parsing, numerals expanded back to number words via `expand_number_token`), and `count_occurrences` counts exact whole-token matches of the target unit (or, for multi-word targets, greedy non-overlapping subsequence matches). Count A is accurate when Whisper faithfully renders every repetition, which holds reliably at low k and *unreliably at high k* – Whisper is itself an autoregressive model and is prone to de-duplicating or collapsing long runs of near-identical speech, exactly the failure mode under study, so it cannot be assumed neutral at the k values that matter most.

Count B (duration-based). Per (model, family, template) cell, a linear fit duration $= a + b \cdot k$ is estimated on the *trusted low-k regime*, defined *before* looking at any high-k behaviour so the calibration cannot be contaminated by the effect being measured: items with $k \leq 3$, Count A exactly equal to the requested k, and audio duration > 0.2 s (`fit_duration_models`). The fit falls back from (model, family, template) to (model, family) to (model) when a cell has too few trusted points, and is discarded if the fitted slope b is not meaningfully positive. Inverting the fit at a given observed duration gives Count B, a count that is *immune* to ASR de-duplication (it never looks at the transcript) but is correspondingly *blind* to a model that babbles for approximately the right length without actually saying the right thing.

Because the two estimators fail in different, largely uncorrelated directions, **correctness is decided conjunctively**, not by picking one estimator or averaging them: an item is labelled `correct` only if Count A exactly equals the expected count *and* the observed duration falls within a tolerance band of the duration the calibrated fit predicts for that expected count (duration_ratio $\in [0.70, 1.45]$, constants `DUR_LO`, `DUR_HI`). The band is wide enough to absorb ordinary prosodic speech-rate variation (a model reading the same words slightly faster or slower is not a counting failure) while still catching the two directions of genuine failure: a transcript that Whisper collapsed but whose audio ran much longer than it should have (loop), and a transcript that is short and whose audio is also short (truncation). Neither estimator is ever used alone to *assert* a count in a disagreement case – the reconciliation only ever *certifies* correctness when both streams of evidence agree; a disagreement produces an `agree=False` audit flag ($|\text{Count A} - \text{Count B}| > \max(1, 0.15 \cdot \text{expected\_count})$) rather than a synthesised number, so that a wrong count can never silently enter the behavioural CSV disguised as ground truth.

4.2 The outcome taxonomy

`classify` in `score_counts.py` assigns exactly one of eight outcome labels, checked in the following fixed order (audio-level degeneracy first, since noise or silence of coincidentally the right duration would otherwise slip through the duration test):

1. `empty` – duration < 0.25 s or RMS $< 10^{-3}$: essentially no audio was produced.
2. `degenerate` – spectral flatness > 0.35 (noise/babble rather than speech) or an empty ASR transcript.

3. `correct` – Count A equals the expected count *and* the duration ratio is in-band (the conjunctive criterion above).

4. `loop` – generation hit the model's token cap, *or* the duration ratio exceeds 1.6: the model kept going well past where it should have stopped.

5. `overcount` – Count A exceeds the expected count (and the item was not already classified as a loop).

6. `truncation` – the duration ratio is below `DUR_LO`: the model stopped early.

7. `undercount` – Count A is below the expected count (and the item was not already classified as a truncation).

8. `miscount` – the residual fallback: none of the above applied, e.g. the transcript count matches but the duration ratio is out of band without meeting either the loop or the truncation threshold ("right count, implausible pacing").

The main paper's Section 4.3 describes this taxonomy at the six-way level a figure caption can carry – {`correct`, `undercount`, `truncation`, `overcount`, `loop`, `degenerate`} – which folds `empty` into `degenerate` conceptually (both are audio-level failures rather than counting failures) and omits the low-frequency `miscount` residual bucket; the eight-way taxonomy above is the one actually implemented and emitted into `data/results/behavioural.csv`'s `outcome` column.

4.3 Audio degeneracy detectors

Computed by `audio_flags` in `src/common/asr_transcribe.py`, independent of the ASR transcript, because a TTS failure does not always manifest as wrong *words* – it can produce noise, a stuck vowel, or silence, none of which a transcript-only metric would catch:

- **RMS** – root-mean-square amplitude of the waveform; near-zero flags near-silence.
- **Voiced fraction / trailing silence** – a smoothed energy envelope (frame length sr/50) is thresholded at 2% of its peak to flag voiced frames; `trailing_silence_s` is the time from the last voiced frame to the end of the clip, distinguishing an abrupt truncation (long trailing silence relative to content) from a clean stop.
- **Spectral flatness** – `librosa`'s spectral flatness measure, near 1 for noise-like spectra and near 0 for tonal/harmonic (speech-like) spectra; used as the `degenerate` threshold above.
- **Loop autocorrelation** – the log-mel envelope (32 mel bands, hop length 256, mean over frequency) is autocorrelated with itself; a strong peak (`loop_ac_peak`) at a lag beyond 0.4 s (`min_lag`) indicates the model is re-rendering materially the same acoustic content periodically – an exact-loop signature distinct from, and complementary to, the duration-ratio-based loop detection in `classify`.

Whisper transcription itself is run with `condition_on_prev_tokens=False`. This is deliberate, not a default left in place: Whisper's own decoder is autoregressive and prone to the same class of repetition lock-in under study, and allowing it to condition on its own previously decoded text would risk the ASR judge's loops contaminating the measurement of the TTS model's loops. Word-level timestamps (`return_timestamps="word"`) are requested for a specific downstream purpose

beyond transcription: mapping an audio-domain repetition boundary back to a decoder step index via step $= t_{\text{word}} \times$ token_rate_hz, where `token_rate_hz` is each model's nominal acoustic-token rate from the panel registry (`src/common/registry.py`: 50.0 Hz for the Llasa family, 21.53 Hz for XTTS-v2, 12.5 Hz for Qwen3-TTS). The registry's own docstring notes that nominal rates are cross-checked against the measured n_{tokens}/duration at smoke-test time, and it is the measured value that downstream analysis actually uses.

4.4 Effective rank, spectral slope, and Kirchhoff index

Computed by `spectral_proxies` in `analysis/state_dynamics.py` on a trajectory $h \in \mathbb{R}^{T\times d}$ of hidden states at one probe layer (mean-centred over time before the decomposition; trajectories longer than `MAX_SVD_STEPS`= 512 steps are uniformly subsampled first, since these are properties of the *shape* of the spectrum, which uniform subsampling preserves, and the alternative is an $O(Td^2)$ SVD per layer per item; trajectories shorter than 24 steps are skipped). Let $\sigma_1 \geq \sigma_2 \geq \cdots$ be the singular values of the centred trajectory matrix and $p_i = \sigma_i / \sum_j \sigma_j$.

- **Effective rank** $\mathcal{N}_{\text{eff}} = \exp\big(-\sum_i p_i \log p_i\big)$: the exponentiated Shannon entropy of the normalised singular-value spectrum, i.e. a participation-ratio-style count of how many singular directions are actually being used, smoothly interpolating between 1 (all variance in one direction) and the nominal matrix rank (perfectly flat spectrum). This is the statistic behind the capacity-gain measurement of Section 4.3 of the main paper.
- **Spectral slope** α: the negative slope of $\log \sigma_i$ regressed against $\log(i+1)$ over the mid-tail index window $i \in [2, \max(6, 0.8 \cdot \text{rank})]$ – excluding the first couple of dominant components and the extreme tail – i.e. the exponent of the spectrum's power-law decay.
- **Kirchhoff index** $\log_{10} Kf = \log_{10}\big(\sum_i 1/\lambda_i\big)$ with $\lambda_i = \sigma_i^2$, floored at $\lambda_i > 10^{-10}\lambda_{\max}$ for numerical stability: the standard graph-theoretic Kirchhoff index (sum of inverse Laplacian eigenvalues, a resistance-distance-based measure of global connectivity) applied to the covariance spectrum of the trajectory, carried over unchanged from the companion Whisper study [1] so its Table 2 and the present study's Table 2 are directly comparable.

4.5 The template bootstrap

Confidence intervals throughout `analysis/capacity.py` are resampled over *stimulus templates*, not over items, and the rationale is stated directly in the source: items sharing a template (e.g. every k-value built from carrier `t1`, "The dog was … big and it ran across the field.") are not independent draws, since they share carrier syntax and length structure; bootstrapping over raw items would understate the true variance. Two statistics are computed this way.

Capacity gain (`gain`): for a given family (repeated or control) and model, $\mathrm{d}\mathcal{N}_{\text{eff}}/\mathrm{d}\log k$ is the ordinary-least-squares slope of $\mathcal{N}_{\text{eff}}$ against $\log k$ at deep probe layers (`layer_frac`> 0.6), fit on items with $k \geq 2$, requiring at least four distinct k values and eight points to attempt a fit at all. The 95% interval is built by resampling the set of templates with replacement 2000 times, refitting the slope on the union of each resample's items every time. A "saturation" value is also reported: the median $\mathcal{N}_{\text{eff}}$ over items in the top half of the observed k range, as a plateau estimate.

Paired gap (`paired_gap`): for every (template, k) cell with both a repeated-item median and a control-item median available, the paired difference (control minus repeated) is computed; because the stimuli were built as matched pairs, this differencing cancels template identity and k exactly, making it the estimator with the least nuisance variance among those available. The 95% interval

bootstraps the *mean* of the vector of paired differences (2000 resamples with replacement), and `frac_pos` reports the fraction of paired differences that are positive, i.e. the fraction of (template, k) cells where the control gained strictly more distinguishable states than its matched repeated twin.

5 The negative result, in detail

This section expands the main paper's Section 4.4 ("A note on what we could not measure"). We consider it a real contribution of the project, not a discarded attempt, and document it at the level of detail a reviewer wanting to check the diagnosis would need. The relevant code is `src/common/boundaries.py` (explicitly marked, in its own module docstring, as *"SUPERSEDED as the paper's primary measurement"*) and `src/common/dispersion.py` (the estimator that replaced it, marked *"the primary state measurement"*).

5.1 What the boundary-difference estimator was

Theorem A is stated about the sequence of *repetition-boundary states* $s_0, s_1, s_2, \ldots$ – the decoder's hidden state exactly at the moment each successive copy of the repeated phrase has been rendered. The most direct way to test the contraction premise is therefore to locate those states in a recorded generation trajectory and difference them. The estimator built to do this, in order:

1. **Locate the k ground-truth text columns.** For a repeated item with target unit u (or a control with k distinct fillers), `unit_columns` walks the model's own tokenized, transformed prompt text left to right with a non-rewinding regular-expression search (`\bword\b`, case-insensitive), using the model-specific `OffsetTokenizer` (`src/common/offset_tok.py`) so that each family's own string transform is respected – notably XTTS-v2, which lower-cases and replaces every space with a literal `[SPACE]` token before tokenizing, so column indices must be (and are) computed on the transformed string, not the raw one. This gives k exact text-span column indices, one per occurrence, by construction in the same left-to-right order for a repeated item and its matched control.

2. **Track an attention centroid.** At the deepest instrumented probe layer, `read_head` computes, for every generation step t, the row-normalised text-attention centroid $c_t = \sum_j j\, p(t, j)$ (a 5-tap moving average is applied for smoothing). A centroid rather than an argmax is used deliberately: the source comment notes that when the k spans are near-identical, their attention columns are near-identical too, so a per-column argmax collapses to the same step for every occurrence, whereas a centroid degrades gracefully and still moves even when no single column dominates.

3. **Match columns to steps monotonically.** `boundary_steps_from_attention` rescales the k target columns onto the empirical 2nd–98th percentile range the centroid trace actually traverses (a centroid, being a weighted mean, is pulled toward the middle of the attended mass and so compresses relative to the raw column range), then walks forward through the centroid trace once, for each rescaled target in turn finding the first step at which the trace first reaches or exceeds it. This is monotone by construction – it can only ever move forward through the generation.

4. **Difference and fit.** The resulting k boundary steps index into the recorded hidden-state trajectory, giving $d_m = \|h(t_{m+1}) - h(t_m)\|$ (`boundary_distances`), and `fit_geometric` regresses $\log d_m = a + m \log q$ by least squares to recover $\hat{q}_{\text{bnd}}$ and an R^2.

5.2 Why it was the natural first choice

This construction is fully self-contained: it uses only the model's own recorded attention and the tokenizer's exact character offsets, with no external ASR pass, no separate alignment model, and no assumption that speech is rendered at a constant tempo. Unlike the estimator that replaced it (Section 5.5), it operationalises Theorem A's boundary states s_m *literally*, at the specific generation step where occurrence m is actually believed to be rendered, rather than approximating the theorem's claim through a proxy quantity. If the model's attention tracked position the way a monotone alignment mechanism would, this would be the right tool, and it is kept in the codebase (rather than deleted) precisely because it remains the natural estimator for any future model that does have a genuinely monotone text read-head.

5.3 The measurement that killed it

The estimator's soundness rests on one load-bearing assumption: that the attention centroid c_t moves *monotonically forward* through the text as generation proceeds, i.e. that it is tracking position, not merely fluctuating around some fixed point of mass. This is directly checkable by computing, over consecutive generation steps, the fraction of steps at which the centroid actually advances ($c_{t+1} > c_t$). A genuine left-to-right sweep would produce a fraction close to 1; a process with no net directional drift – a random walk – would produce a fraction close to 1/2.

The measured value, reported in the project's running notes (`findings.md`) and in the docstrings of both `dispersion.py` and `boundaries.py`, is that **the deep-layer attention centroid advances on approximately** 51% **of generation steps** – statistically indistinguishable from an unbiased random walk, and nowhere near the sweeping behaviour the estimator needs. (We note for precision that this figure is recorded as an empirical summary in the project's notes and source comments rather than reproduced by a single standalone script in the current snapshot of `analysis/`; it is the stated empirical justification, cited three times independently across the codebase, for abandoning the estimator, and it is corroborated by the concrete symptom described next, which *is* directly reproducible from committed data.)

That concrete symptom is visible in `data/results/pooled_q.csv` (produced by `analysis/pooled_q.py`, which pools boundary-distance curves across items within a (model, family) cell and regresses $\log(d_m/d_0)$ on m through the origin, bootstrapping the item set for a confidence interval). Under the boundary/attention method, the pooled fit for Llasa-1B recovers $\hat{q}$ statistically indistinguishable from 1 in every instrumented family – `control_word`: $\hat{q} \approx 1.00$ (95% CI $[0.99, 1.02]$); `sentence_rep`: $\hat{q} \approx 1.03$ (95% CI $[1.00, 1.12]$); `word_rep`: $\hat{q} \approx 0.98$ (95% CI $[0.94, 1.02]$) – i.e. *no measurable contraction at all*, for a model whose boundary-free dispersion estimate (Section 5.5) recovers $\hat{q}$ clearly below 1 for the same generations. This is a snapshot from the instrumented panel available at the time of writing and individual decimal values will move as more of the panel finishes running; the qualitative pattern – boundary-method $\hat{q} \approx 1$ versus dispersion-method $\hat{q} < 1$, on the identical underlying trajectories – is the diagnosis, not the specific digits.

5.4 Why the failure is self-defeating by construction

The failure is not a matter of insufficient smoothing or a fixable implementation bug; it is a direct structural corollary of Lemma B (Section 1.2). `softmax_dist_le` proves that the per-occurrence attention-weight differences shrink as $O(1/k)$, and `entropy_ge` proves the profile's entropy floor grows as $\log k - \delta$: as the repeated block grows, the attention distribution the estimator's centroid is computed from becomes provably closer to uniform. A weighted-mean centroid computed over

an increasingly uniform distribution carries, by the same argument, increasingly little information about which position is truly being rendered – in the limit its expected value converges to the midpoint of the attended span regardless of which occurrence is actually current. The estimator therefore degrades precisely as k grows, which is exactly the regime in which a clean boundary estimate would matter most for testing the theorem. This is why the failure could not have been patched by using more attention heads, denoising the centroid trace, or any comparable engineering fix: the very effect the estimator was built to exploit (identifiable, position-informative attention) is the effect Lemma B proves vanishes as the block it is trying to measure grows.

5.5 The replacement: boundary-free dispersion

Theorem A(ii) (`exists_indistinguishable`, Section 1) licenses a different, boundary-free operationalisation: it says the states visited late in a repeated generation become mutually indistinguishable as a *set*, which is a statement about the *spread* of the visited states and needs no per-occurrence boundary labels at all. `src/common/dispersion.py` implements this directly: the trajectory is tiled into `N_WINDOWS`= 8 equal-length windows (each requiring at least `MIN_WINDOW`= 8 steps to support an estimate); within each window, states are ℓ_2-normalised (direction, not residual-stream magnitude, is what is compared – magnitude is close to constant along these trajectories in any case, roughly 130 for Llasa-1B, so normalising changes little but makes the resulting quantity comparable across models with different activation scales) and the mean pairwise chordal distance $\sqrt{2 - 2\cos\theta}$ among normalised states in the window is computed; $\log \sigma(\text{window})$ is then fit linearly against window index, and the fitted per-window rate is converted to a per-repetition $\hat{q}$ by the ratio of window length to average repetition length (n_{windows}/k). This sidesteps the localisation problem entirely, at the cost of measuring a slightly different (but, per Theorem A(ii), equally licensed) quantity: the spread of the states visited within a window of the trajectory, rather than the literal distance between two specific, individually-identified boundary states. `analysis/state_dynamics.py` computes both estimators side by side for every instrumented item (columns `q_hat` for dispersion and `q_bnd` for the boundary method in `data/results/state.csv`) and records which localisation method, `attention` or the uniform-tempo fallback `uniform`, produced each row's boundaries, precisely so this substitution is auditable rather than silent.

6 Per-model implementation notes

Table 6 reproduces the model panel (`src/common/registry.py`) in full; the notes below detail how instrumentation was obtained for each family and, for Qwen3-TTS, precisely what could not be. The panel is exactly these six checkpoints. The registry also carries a seventh entry, `xtts2norp` – XTTS-v2 with its shipped repetition penalty disabled – which is an ablation of the `xtts2` row, not an independent seventh checkpoint, and is deliberately omitted from Table 6 for that reason; every panel-pooled statistic in the main paper and in this document excludes it by construction (`ABLATIONS = {"xtts2norp"}` in `analysis/make_numbers.py`, `analysis/capacity.py`, `analysis/capacity_confound.py`, and `analysis/unit_invariance.py`). The ablation is documented on its own terms in Section 8.5.

`probe_layer_indices` always force-includes the final layer of whichever density setting is used, since that is the layer that actually feeds the model's own readout (stop-token logit or codec head); the "every n" densities on the larger checkpoints exist to bound per-item activation storage as depth grows.

Table 6: Model panel (`PANEL`, `src/common/registry.py`).

Key	HF id	Fam.	Par.	Token rate	Probes	Notes
llasa1b	HKUSTAudio/Llasa-1B	llasa	1B	50.0 Hz	all	Llama-3.2-1B / X-codec2
llasa3b	HKUSTAudio/Llasa-3B	llasa	3B	50.0 Hz	every 2	Llama-3.2-3B, same tokenizer/codec
llasa8b	HKUSTAudio/Llasa-8B	llasa	8B	50.0 Hz	every 4	Llama-3.1-8B, top of scale ladder
xtts2	coqui/XTTS-v2	xtts	0.4B	21.53 Hz	every 3	GPT2-style AR + HiFiGAN
qwen06b	Qwen/Qwen3-TTS-12Hz-0.6B-Base	qwen	0.6B	12.5 Hz	every 3	dual-track multi-codebook LM
qwen17b	Qwen/Qwen3-TTS-12Hz-1.7B-Base	qwen	1.7B	12.5 Hz	every 3	same family as 0.6B

6.1 Llasa (1B / 3B / 8B)

`src/models/llasa_gen.py` runs every Llasa checkpoint through the same two-pass design. **Pass 1 (generate)** samples speech tokens autoregressively via `model.generate` with the attention backend set to SDPA (`do_sample=True`, temperature 0.8, top-p 1.0, `max_new_tokens`= 2048 by default); the prompt is built from the model's own chat template with sentinel tokens (`<|TEXT_UNDERSTANDING_START/END|>`, `<|SPEECH_GENERATION_START/END|>`) and `continue_final_message=True` to prime the assistant turn on the speech-start token; generated speech-token ids are extracted from the `<|s_NNN|>` vocabulary pattern and saved as integer arrays, with waveform synthesis deferred to a separate offline pass (`src/models/xcodec2_decode.py`) run in the isolated `xcodec2` environment (Section 7). A `hit_cap` flag records whether generation ran to the token ceiling; per the project's running notes, 21.5% of Llasa-1B generations do so, treated in this study as a signal (runaway generation is common enough to be informative) rather than noise to be tuned away by simply raising the cap.

Pass 2 (instrument) runs only when the seed equals `instrument_seed` (default 0), the stimulus item is marked `instrumented`, and at least one speech token was produced. The *entire realised sequence* (prompt followed by the tokens actually sampled in Pass 1, EOS stripped) is fed through a single teacher-forced forward call with the attention backend switched to `eager` – the only backend that materialises per-layer attention-weight tensors – and `output_hidden_states=True`. Because the model is causal, position t of this forward pass carries exactly the hidden state that *produced* generated token t during the actual sampling pass, so this single clean forward recovers the whole instrumented trajectory without altering the realised token sequence in any way; it is a pure read-out of Pass 1's output, not a re-sampling. `TextAttentionRecorder` attaches forward hooks to a chosen subset of `self_attn` layers (up to four of $\{0, \lfloor L/3 \rfloor, \lfloor 2L/3 \rfloor, L-1\}$ for L total layers); each hook slices the returned `(attn_output, attn_weights)` tuple's weights down to `[B, H, Q, text_lo:text_hi]`, head-averages, and moves to CPU `float16` *inside the hook*, specifically to avoid ever materialising the full $O(T^2 \cdot \text{heads} \cdot \text{layers})$ attention tensor in memory. Entropy and top-1 probability of the next-token distribution are recorded from the same forward pass's logits at no extra cost. `llasa_gen.py`'s own comment states the reason SDPA is used for sampling and eager only for the instrumented pass directly: Llasa under eager attention is roughly 5× slower to sample.

6.2 XTTS-v2

`src/models/xtts_gen.py` generates via `model.gpt.generate` (the GPT2-style autoregressive mel/codec-index decoder), using the sampling defaults shipped in the checkpoint's `config.json` unless overridden on the command line. Those shipped defaults include `repetition_penalty`= 5.0 – confirmed directly in the downloaded checkpoint's `config.json` – an aggressive built-in anti-repetition mechanism active during the very sampling pass this study measures. Any residual counting failure XTTS-v2 exhibits is therefore occurring *despite* a penalty actively working against it, which is why the main paper's Discussion (Limitations) frames XTTS-v2 as a *conservative*, not a neutral, member of the panel: if anything, the theory's prediction is being tested on this model under a handicap. The AR decoder's built-in generation ceiling (`model.gpt.max_gen_mel_tokens`) is derived from the shipped config's `model_args.gpt_max_audio_tokens`= 605 minus a small fixed number of reserved special-token positions, giving an effective cap of approximately 602 mel frames per item; the script's own `--max-new-tokens` override can only shrink this further via `min()`, never raise it.

Instrumentation avoids a second forward pass through the high-level API by hand-reconstructing XTTS's own internal embedding recipe: conditioning latents, text embeddings (token + positional), and mel-token embeddings (token + positional) are concatenated into a single `inputs_embeds` tensor and fed through the *raw* underlying `transformers.GPT2Model` (`model.gpt.gpt`, whose own `wte`/`wpe` embedding tables are deleted by XTTS since embeddings are computed externally). This bypasses `Xtts.inference()`/`synthesize()` specifically because those high-level entry points do not expose `output_hidden_states=True` and additionally run a sentence-splitting/chunking stage that could silently fragment a long repeated-word stimulus into several independent `generate()` calls, defeating the "one contiguous generation" premise the state-dynamics measurement depends on. Attention is forced to `eager` once at load time (`model.gpt.gpt.config._attn_implementation = "eager"`), because SDPA (the library default) silently returns `None` attention weights even when `output_attentions=True` is requested – a failure mode the script's header comment documents explicitly, verified against the installed `coqui-tts` 0.27.5 and `transformers` 4.57.1 sources by file and function name. The same memory-avoidance trick as Llasa is used (`output_attentions=False` at the top-level call, so `GPT2Model` never collects the full attention stack across all layers, while per-layer forward hooks on the probed blocks' `.attn` submodules see the real per-layer weights regardless, since `transformers`' eager attention path computes them unconditionally internally). Vocoding runs in the same process and script as generation (unlike Llasa, which vocodes offline in a separate environment), since XTTS ships its own HiFiGAN decoder with no conflicting pinned dependency.

6.3 Qwen3-TTS (0.6B / 1.7B)

`src/models/qwen_gen.py` instruments a model that is architecturally very different from the other two families. Qwen3-TTS is a dual-track model: an autoregressive "talker" transformer (`Qwen3TTSTalkerForConditionalGeneration`) emits one 12.5 Hz acoustic frame per decode step, where each frame packs 16 RVQ codebook ids – the first sampled directly by the talker's own `codec_head` inside the standard HF generation loop, the remaining 15 filled in afterward by a separate, smaller "code predictor" sub-model conditioned on that step's talker hidden state. Only the first-codebook / talker level is instrumented, which is also the level at which Theorem A's boundary-state argument is naturally stated.

No second forward pass is needed. The mid-level library API, `Qwen3TTSForConditionalGeneration.generate()`, already unconditionally calls the talker's

own `.generate(..., output_hidden_states=True, return_dict_in_generate=True)` internally on every call and then discards every hidden-state layer but the last. `TalkerGenerateCapture` monkeypatches the bound `talker.generate` method to stash the full raw `GenerateDecoderOnlyOutput` (and the `trailing_text_hidden` keyword argument) before forwarding the call through completely unchanged – a read-only interception, so sampling and behaviour are byte-for-byte identical to what the public `generate_voice_clone()` API would have produced with or without instrumentation. Per-frame, per-probe-layer hidden states are then pulled out of the captured object after the fact (`extract_trajectory`), with frame j aligned to the decode call whose forward pass predicted it, matching the library's own internal step/frame convention.

Text attention is not obtainable, and this is stated in the code as an architectural fact, not a shortfall of instrumentation effort. In the model's default streaming mode, the embedding fed into the talker at each decode step is the *elementwise sum* of a codec-token embedding and, for a short prefix of steps, a text-token embedding (`trailing_text_hidden`, added inside the talker's own `forward`). There is consequently no column of the self-attention matrix that corresponds to "the text" the way there is for Llasa's or XTTS's concatenated token streams: attention weight over a fused position cannot be attributed to its text component versus its codec component after the sum, because the sum is not invertible. A `non_streaming_mode=True` flag exists in the library that would give text its own contiguous block of positions and could in principle be probed, but it is not the model's default or tested code path, and using it risks degrading the audio quality the project treats as the non-negotiable baseline requirement, so it is deliberately not used. `text_lo`/`text_hi` are therefore written as literal `null` in this model's metadata records – an explicit "not applicable," never a fabricated or uniform placeholder span standing in for real boundaries – alongside a `trailing_text_len` field recording how many initial decode steps did carry a distinguishable text contribution, offered as a temporal (not spatial) proxy for anyone who wants one. A direct consequence, made explicit here because it is easy to miss: because Qwen3-TTS has no obtainable text attention, it is absent from the boundary-based estimator of Section 5 *by construction* (there is no attention tensor to feed `unit_columns`/`read_head`), and its state-space measurements in this study are dispersion-only.

7 Reproducibility

7.1 The four conda environments

`env/setup_envs.sh` builds four environments because, per its own header comment, "the dependency constraints are mutually unsatisfiable." Table 7 lists every version pin found in the script and the concrete failure it exists to prevent, drawn from the script's own comments and the README's "Things that will bite you" section.

The `qwen` and `coqui` environments both install torch and torchaudio from the PyTorch `cu130` wheel index (`--index-url https://download.pytorch.org/whl/cu130`) *before* the TTS package itself, per the script's own top-level comment: "install cu130 torch BEFORE the TTS package in each env, else pip silently pulls a default CUDA-12 wheel and the driver/toolkit combo breaks."

7.2 Other pitfalls documented in the README

- **`src/common/tokenizers.py` must never exist.** Running a script from inside `src/common/` puts that directory first on `sys.path`, so a file with that name would shadow the real

Table 7: Conda environments (`env/setup_envs.sh`) and their pins.

Env	Python / core	Pin and the failure it prevents
`base`	py3.13, torch 2.11+cu130	`transformers==4.57.3` (additive install onto the existing base torch); runs the Llasa LMs, Whisper ASR, and all analysis scripts.
`qwen`	py3.12	`qwen-tts` package; torch/torchaudio installed from the `cu130` index *before* `qwen-tts` – installing in the other order lets pip silently pull a default CUDA-12 wheel, breaking the driver/toolkit match.
`coqui`	py3.12	`coqui-tts`; same cu130-before-package ordering. `coqui-tts` breaks on `transformers` 5.x (`isin_mps_friendly` was removed from that version), so this environment is kept on `transformers` 4.57.x.
`xcodec2`	py3.11, torch 2.5 (hard-pinned)	`xcodec2==0.1.5` pins torch 2.5 but leaves `transformers`/`torchao` unpinned, and both have since moved past compatible versions: `transformers` 5.x → `Could not import module 'PreTrainedModel'`; `torchao` $\geq$0.7 → `module 'torch' has no attribute 'int1'`. The working combination, pinned explicitly after the `xcodec2` install, is `transformers==4.46.3`, `tokenizers<0.21`, `torchao==0.6.1`, plus `soundfile`.

`tokenizers` package and break `transformers` with a confusing `ImportError`; this is why the project's own tokenizer interface is named `offset_tok.py` instead.

- **The HF `automatic-speech-recognition` pipeline routes audio through `torchcodec`**, which does not load against the installed FFmpeg in this environment; `asr_transcribe.py` drives `WhisperProcessor` and `WhisperForConditionalGeneration` directly instead, avoiding that code path entirely.
- **XTTS replaces spaces with literal `[SPACE]` tokens and lower-cases before tokenizing**, so any text-column index computed for XTTS must be computed on the transformed string (`offset_tok._xtts_prepare`), never on the raw stimulus text.
- **Llasa under eager attention is roughly 5× slower to sample**, which is why `llasa_gen.py` samples under SDPA and switches to eager only for the single instrumented teacher-forced pass.
- **Whisper de-duplicates repeated speech**, so a transcript-only repetition count systematically understates loops; this is the direct motivation for the conjunctive correctness criterion of Section 4.

7.3 Exact commands

```
# 0. Environments (once)
bash env/setup_envs.sh all       # base, qwen, coqui, xcodec2
bash scripts/setup_lean.sh       # elan + mathlib cache (CPU, ~15 min)
bash scripts/download_models.sh  # 8 checkpoints, ~74 GB
```

```
# 1. Verify the formal artifact
bash scripts/check_lean.sh

# 2. Generate (one model per GPU; resumable by (item_id, seed))
python src/models/llasa_gen.py --model llasa1b --gpu 0 --seeds 0 1 2
# xtts2 needs the `coqui` env and COQUI_TOS_AGREED=1
# qwen06b / qwen17b need the `qwen` env

# 3. Vocode, transcribe, score
bash scripts/run_pipeline.sh <asr_gpu> llasa1b xtts2 ...

# 4. Analyse and build the paper
python analysis/state_dynamics.py --models <models> --out data/results/state.csv
python analysis/capacity.py                        # the central measurement
python analysis/horizon_fit.py                      # behavioural collapse points
python analysis/figures.py
bash paper/build.sh                                 # regenerates numbers, compiles
```

`paper/build.sh` regenerates `numbers.tex` from the result CSVs before every compile (`analysis/make_numbers.py`), so a number in the compiled PDF cannot drift from the data it was computed from; every generation script is resumable, skipping any `(item_id, seed)` pair already present in its metadata file, so re-running after an interruption is always safe. Because generation and analysis for this panel were still in progress at the time this document was written, several of the per-model quantities in the main paper's Table 1 and Table 2 are recomputed automatically the next time `paper/build.sh` runs against a more complete `data/results/` directory; this document has deliberately favoured describing method over quoting point-in-time numbers for exactly that reason. Section 8 inherits the same discipline, and in one case (Section 8.2) reports a concrete instance of exactly this kind of point-in-time drift, caught by re-running an unmodified analysis script against the current `data/results/` directory.

7.4 The verification gate

`scripts/check_lean.sh` is reproduced in full in Section 1.3. Its three checks – successful `lake build`, a textual `sorry` scan, and the `#print axioms` audit for `sorryAx` – are run as a single command, `bash scripts/check_lean.sh`, and constitute the project's gate on the formal artifact: no result in the main paper or in this document should be read as resting on a machine-checked guarantee unless this script has been run against the exact commit being cited and has printed `PASS: formal artifact verified`.

8 Answering the alternatives

Two rounds of review produced five distinct, falsifiable alternative explanations for the main paper's results – not stylistic objections, but specific mechanisms that, if true, would mean the paper is measuring something other than what it claims. Each is answered here with a new measurement rather than a rebuttal in prose, following the same discipline as the rest of this document: state the objection the way a reviewer stated it, state precisely what would have to be true in the data for the alternative to explain the main result, describe the test and why it is the right test for that alternative specifically, give the numbers as they appear in the committed result file, and say plainly where the evidence is strong and where it is not. Section 8.6 then collects the statistical machinery used throughout – Wilson intervals, the template bootstrap, and an explicit, uncorrected count of the comparisons made – in one place.

8.1 Naturalness: is the dissociation just improbability?

The objection. Section 3 constructs every repeated item's control by replacing k copies of one word with k distinct fillers from the same template's pool. A skeptical reading of the main paper's Figure 1(b) dissociation does not need periodicity at all: twelve identical copies of *very* is a far more out-of-distribution string, under any reasonable model of English, than twelve of *quite, really, truly, fairly, …* strung together in the same carrier. If a decoder's failure tracks how unlikely its input is rather than how periodic it is, the paper's sharpest test would be confounding the two variables it claims to have separated.

What would have to be true. For naturalness to explain the gap, the repeated item in a matched pair would have to be the *more* improbable member – lower likelihood, higher per-token surprisal – under a competent, independent model of English, and this would have to hold broadly across the 54 matched pairs the dissociation is built from, not just in a handful of cases.

Test design. `analysis/text_perplexity.py` scores every stimulus's raw text under a causal LM's per-token cross-entropy (`model(ids, labels=ids).loss`, which Hugging Face already averages over tokens) and compares each `control_word` item against the `word_rep` item named in its `control_of` field – the identical 54 matched pairs used throughout Section 3. Its first run used `HKUSTAudio/Llasa-1B` as the scorer, on the pragmatic grounds that the checkpoint was already local; a reviewer correctly pointed out that this is not a neutral referee, since Llasa-1B's backbone (Llama-3.2) is literally one of the panel's own architectures, and a naturalness check that answers "is this text unlike what a member of the tested family expects" cannot rule out the panel's own inductive bias leaking into the measurement. `analysis/naturalness_independent.py` reruns the identical per-token-NLL procedure under `microsoft/phi-2` (2.7B) as the primary scorer – a different lab, a different training recipe, and, critically, a different architecture family from all three panel backbones (Llama-3.2 for Llasa, the GPT-2-style AR decoder for XTTS-v2, Qwen3 for Qwen3-TTS), so no panel member's failure can be attributed to sharing an inductive bias with the referee. `gpt2-large` is reported alongside it as a secondary cross-check only, because XTTS-v2's own AR decoder is itself a GPT-2-style architecture; a GPT-2-family LM is therefore not fully independent for that one panel member, and the paper's naturalness claim never rests on it alone. Confidence intervals on the mean paired gap are built by resampling the six stimulus *templates* with replacement, not the 54 pairs, for the reason given in Section 4 and restated in this script's own docstring: the 54 pairs are 6 templates $\times$ 9 values of k, so pairs sharing a template are correlated draws and treating all 54 as independent would understate the true sampling variance.

Numbers. Table 8 reports all three scorers. Under the panel-backbone scorer (`data/results/text_nll.json`), the control is the less probable member of the pair in 89% of the 54 pairs (mean per-token cross-entropy 16.05 for controls vs. 14.85 for repeated items, gap 1.20 nats). Under the independent primary scorer, phi-2 (`data/results/text_nll_independent.json`), the control is still the less probable member in 87% of the 54 pairs (4.89 vs. 3.69, gap 1.20 nats, 95% template-bootstrap CI $[0.58, 1.67]$). The secondary cross-check, gpt2-large, agrees even more strongly: the control is less probable in every one of the 54 pairs (gap 1.98 nats, CI $[1.80, 2.17]$) – a Wilson interval on that 54/54 proportion is $[93.4, 100.0]$% (Section 8.6), which is the cleanest illustration in this project of why Wilson rather than the normal approximation is used: a normal interval around $p = 1$ is degenerate and would have to be truncated by hand, while Wilson returns a legitimate bounded interval directly.

The disagreement, stated honestly. All three scorers agree on the headline direction and on the pooled magnitude of the gap ($\approx$ 1.2–2.0 nats, control less probable in 87–100% of pairs). They do *not* agree on the shape of the gap across k, and the main paper's text is deliberately worded to claim only what both the panel-backbone and the independent scorer support. Under

Table 8: Naturalness check, three scorers, 54 matched pairs. "Ctl. higher" is the fraction of pairs in which the control has the higher (less probable) per-token cross-entropy.

Scorer	Role	Rep. mean	Ctl. mean	Gap	Ctl. higher
Llasa-1B	panel backbone	14.85	16.05	1.20	89%
phi-2	independent, primary	3.69	4.89	1.20	87%
gpt2-large	secondary cross-check	3.36	5.34	1.98	100%

Llasa-1B, the per-k gap widens monotonically across the entire ladder, from 0.37 nats at k=2 to 1.95 nats at k=32 (`by_k` in `text_nll.json`). Under phi-2, the gap widens similarly at first – 0.34 nats at k=2, rising to a peak of 2.62 nats at k=8 – but then *narrows*: 2.12 at k=12, 1.45 at k=16, 0.24 at k=24, and by k=32 it has reversed sign to -0.62 nats, with the fraction of pairs in which the control is less probable falling from 100% at k=3–12 to 33% at k=32 (`by_k_frac_ctl_higher` in `text_nll_independent.json`). At the very top of the ladder, in other words, phi-2 finds 32 verbatim copies of one word *easier* to predict than the length-matched control, which is plausible on its own terms – a small, locally-conditioned LM's induction-head copying becomes close to trivial once a token has already repeated a dozen times, whereas the control at k=32 is 32 fillers cycled four times through an eight-word pool, which is repetitive in a different, less exploitable way. This reversal sits entirely past every panel checkpoint's own k^* (1–8, Table 1 of the main paper): by the k range where counting collapse actually happens, both scorers agree without qualification. The paper's claim is accordingly the conjunction, not the union, of what the two scorers show: the control is the less probable member of the pair in the large majority of cases and across the range where the phenomenon occurs, and naturalness therefore runs opposite to, not in support of, the alternative explanation. A claim that the gap *widens monotonically with* k, which only the non-independent scorer supports, is not made.

Conclusion. Naturalness does not explain the dissociation. If anything, the harder-to-predict member of each pair – the control, under every scorer used – is the one the panel renders correctly, which is the reverse of what the alternative needs. The evidence for the headline direction is strong (three scorers, two of them mutually independent, agree); the evidence for any particular trend of the gap across k is weak past the point where collapse has already occurred, and the paper does not lean on it.

8.2 Unit invariance: repetitions, or acoustic tokens?

The objection. A second alternative locates the attractor on the acoustic side entirely, with text playing no causal role: perhaps a decoder simply loses its place once it has emitted enough near-identical codec frames in a row, regardless of why those frames are near-identical. Under this account, "periodicity" is doing no work beyond producing repetitive audio, and the same collapse should occur for any input that yields enough near-duplicate acoustic tokens, text-side repetition structure notwithstanding.

What would have to be true. A whole repeated sentence costs several times more acoustic tokens per repetition than a single repeated word. An acoustic-token-budget account therefore predicts that sentence repetition should collapse at a substantially *smaller* k than word repetition (fewer repetitions are needed to accumulate the same token budget), and that the *token count* at the collapse point should be roughly equal across the two families. Theorem A predicts the

opposite on both counts: collapse at a comparable number of repetitions regardless of unit, and a token count at collapse that differs in proportion to the unit's cost.

Test design. `analysis/unit_invariance.py` computes, per model, k^* separately for `word_rep` and `sentence_rep` using the same chance-adjusted threshold as the main paper's Table 1 (largest k with accuracy > 0.5), together with the median number of realised acoustic tokens (`n_speech_tokens`) at that k. Comparing k^* directly tests the repetition-count account; comparing the token count at k^* tests the token-budget account; running both on the same models from the same behavioural table lets the two predictions be checked against each other rather than against different data.

Numbers. Table 9 reproduces `data/results/unit_invariance.json` as committed: mean $|k^*_{\text{word}} - k^*_{\text{sent}}| = 1.7$ repetitions across 6 models, against a mean token-count ratio at collapse of $1.34\times$ – repetitions differ by less than two units while tokens differ by a third, which is the pattern Theorem A predicts and the token-budget account does not.

Table 9: Unit invariance, as committed in `data/results/unit_invariance.json`. Token counts are medians at the reported k^*; `n/a` where no k cleared the 0.5 threshold.

Model	k^*_{word}	tok. (word)	k^*_{sent}	tok. (sent)
Llasa-1B	0	n/a	3	351
Llasa-3B	1	209	3	457
Qwen3-TTS-0.6B	4	53	4	80
Qwen3-TTS-1.7B	8	80	4	86
XTTS-v2	4	89	4	128
XTTS-v2 (no penalty, ablation)	2	63	1	29

A data-integrity caveat, found while verifying this section. The committed `unit_invariance.json`'s six-model set is {`llasa1b`, `llasa3b`, `qwen06b`, `qwen17b`, `xtts2`, `xtts2norp`} – it includes the `xtts2norp` ablation and omits Llasa-8B entirely, in tension with the rule stated in Section 6 and enforced everywhere else in this project's pipeline that the ablation is never pooled as if it were an independent checkpoint. The cause, verified directly: at the time this file was written, `data/results/behavioural.csv` carried only 319 of 540 expected rows for Llasa-8B (59%), and for at least one of its two families (`word_rep` or `sentence_rep`) accuracy had not yet cleared the 0.5 threshold at any k, so `unit_invariance.py`'s own finiteness check dropped it from the output, leaving `xtts2norp` as the sixth entry by coincidence of ordering rather than by design – the script itself (re-inspected for this section) does correctly exclude `ABLATIONS` by construction, so this is a snapshot-timing issue in the committed artifact, not a logic error in the analysis code. Re-running the identical, unmodified script against the current `data/results/behavioural.csv` – performed here for verification, and not written back to the repository, since this document's brief is to report on committed results rather than to regenerate them – gives the true six-panel-member set with Llasa-8B in place of the ablation: k^*_{word}=4 (median 328 tokens), k^*_{sent}=3 (median 424 tokens) for Llasa-8B, and an across-panel mean $|k^*_{\text{word}} - k^*_{\text{sent}}|$ of 1.7 repetitions (unchanged to one decimal) against a mean token ratio of $1.50\times$ (vs. the committed file's $1.34\times$). The qualitative conclusion is unaffected by the correction – repetitions, not tokens, is still the unit that predicts the collapse point – but the committed file, and the 6-model statistic quoted from it in the main paper, should be regenerated once Llasa-8B's behavioural generation completes, and the corrected number should read as a true six-of-six-panel statistic rather than a six-of-seven-including-the-ablation one.

Conclusion. The horizon is counted in repetitions, not accumulated acoustic tokens: k^* moves by under two repetitions between a unit that costs one token and one that costs several times more,

while the token count at collapse tracks the unit's cost almost exactly. The evidence is qualitatively robust to the data-integrity issue just described; the point estimate of the token-count ratio is not, and should be treated as provisional until the artifact is regenerated. One further per-cell caveat: Llasa-1B's word-repetition accuracy never exceeds 0.5 even at k=1 (Table 9, `n/a`), so no token count at collapse is defined for that cell; the mean token-count ratio is accordingly computed over five models for the word side of the comparison, not six, a distinction the committed JSON's flat `mean_tok_ratio` field does not surface on its own.

8.3 Capacity confound: is effective-rank decline tautological?

The objection. A model that correctly renders *very* sixteen times in a row produces audio that genuinely is more acoustically monotonous than a control rendering sixteen distinct words. A shrinking effective rank on repeated-text trajectories could then be measuring nothing but this surface fact about the stimulus – the audio the prompt asked for – rather than any loss of the decoder's capacity to represent *how many* repetitions have occurred. Under this reading, the main paper's capacity-gain ratio (0.50, Section 4.3) would be a restatement of the stimulus design, not a discovery about the decoder.

What would have to be true. For the tautology objection to fully account for the ratio, the repeated-vs-control gap in effective-rank growth would have to close once (i) the trajectory's own realised output diversity is held constant by regression, so "more repetitions requested" is separated from "less varied audio produced," and (ii) the comparison is restricted to items the model rendered *correctly*, where the audio is by construction exactly what was asked for and no degenerate-output confound can apply.

Test design. `analysis/capacity_confound.py` implements both checks on data already collected, per model and per family (`word_rep`, `control_word`). The *adjusted* test regresses $\mathcal{N}_{\text{eff}}$ on $\log k$ together with three z-scored covariates of the emitted acoustic-token sequence: the type/token ratio, the immediate-repeat rate (the fraction of tokens equal to their immediate predecessor, a direct index of local acoustic monotony), and the raw token count; the reported quantity is the $\log k$ coefficient with the covariates included, i.e. the capacity gain that remains after realised output diversity is partialled out. These covariates are only available for checkpoints whose acoustic-token ids are persisted to disk, which in this panel is the Llasa family only – XTTS-v2 and Qwen3-TTS contribute a token count but not the distinct-token identities needed for type/token ratio or repeat rate, so their adjusted cells are inestimable rather than zero. The *correct-only* test refits the same unadjusted regression restricted to items labelled `correct` at seed 0 (Section 4); coverage thins sharply at high k precisely because correct renderings become rare there, which is the phenomenon under study rather than a defect of the test, so per-model n is reported alongside every ratio rather than suppressed.

Numbers. Table 10 reproduces `data/results/capacity_confound.json`'s per-model ratios (repeated slope over control slope; smaller means the repeated-text trajectory gains fewer distinguishable states per doubling of k than its control). The panel median is 0.50 unadjusted (n=6 checkpoints), 0.50 after controlling for output diversity (n=3, the Llasa sub-family only, for the reason above), and 0.47 restricted to correct-only renderings (n=5).

The three medians agree to within two hundredths of each other, which is the central result: neither control moves the ratio meaningfully away from the unadjusted panel figure. Two per-model cells deserve a direct flag rather than being folded silently into the median. Llasa-1B's correct-only ratio, 0.01, looks like the tautology objection winning outright for one checkpoint, but it rests on 14 repeated-side and 32 control-side items – a small-sample cell in which the repeated-side slope is itself measured at 0.47, i.e. essentially flat, and a single noisy near-zero slope in the denominator's

Table 10: Capacity-confound ratios (repeated/control slope of $\mathcal{N}_{\text{eff}}$ against $\log k$), from `data/results/capacity_confound.json`. n pairs are (word_rep, control_word) item-layer rows entering each regression; `---` marks an inestimable cell (fewer than 12 usable rows, or fewer than 4 distinct k values).

Model	Raw	+diversity	correct-only
Llasa-1B	0.42	0.50	0.01 (n=14/32)
Llasa-3B	0.28	0.34	0.80 (n=24/22)
Llasa-8B	0.43	0.75	— (n=0/0)
Qwen3-TTS-0.6B	0.57	—	0.31 (n=42/84)
Qwen3-TTS-1.7B	0.78	—	0.47 (n=64/82)
XTTS-v2	0.60	—	0.56 (n=22/41)
Panel median	**0.50**	**0.50**	**0.47**
XTTS-v2 (no penalty, abl., not pooled)	0.89	—	— (n=9/22)

counterpart is enough to send the ratio close to zero without implying the gap has actually closed; read this cell as unstable, not as evidence the effect vanishes. Llasa-8B's correct-only cell is empty (n=0): none of its seed-0, $k \geq 2$ `word_rep` items were labelled `correct`, consistent with the low k^* reported for that checkpoint elsewhere in this document (Section 8.2) rather than with any failure of the confound test itself.

Conclusion. The tautology objection is answered, with two honest qualifications. Holding realised output diversity fixed and restricting to demonstrably correct renderings both leave the panel-median ratio within rounding of the raw figure (0.50 raw vs. 0.50 adjusted vs. 0.47 correct-only), so the capacity-gain gap is not an artifact of repeated audio being acoustically monotonous by construction. The qualification: the diversity-adjusted control is well-powered for only three of the six panel checkpoints, because acoustic-token identities are persisted for the Llasa family alone, so 0.50 is a narrower claim than the raw, six-checkpoint 0.50; and the correct-only control has two unstable or empty per-model cells at the extremes of the panel's capability range, so it should be read as directionally corroborating the raw result rather than as an independently decisive test in its own right.

8.4 The competing account: does the count survive in the representation?

The objection. Venkatesh [2] reports that in text-only autoregressive LMs performing repeated-token counting, a linear probe on the residual stream decodes the true count near-perfectly at every layer, including the exact layer where the model's own output is already wrong: the count is not erased, only unused by the output pathway. A trained linear probe is precisely the L-Lipschitz readout class Theorem A(iii)/(iv) (main paper, Section 2) constrains, so if the same dissociation holds for TTS decoders, the effective-rank account of Section 4.3 would be measuring the wrong thing: a small, count-specific subspace could persist and remain linearly readable inside a trajectory whose *aggregate* rank still looks low, since a whole-trajectory participation-ratio statistic like $\mathcal{N}_{\text{eff}}$ has no reason to be sensitive to a low-dimensional signal buried inside it.

What would have to be true. For the competing account to hold here, a linear probe would have to recover k about as well from the *late* part of a repeated-text trajectory as from the *early* part – no within-trajectory degradation – and about as well on repeated text as on the length-matched control. Representational degradation, by contrast, predicts within-trajectory loss specific to periodic conditioning: better decodability early (when the boundary between "count so far" and "count remaining" is still information the decoder has just read off the text) than late

(once Theorem A(ii)'s indistinguishability has had repetitions to bite), and no comparable loss on the non-repetitive control.

Test design. `analysis/probe_count.py` fits a ridge regression predicting $\log_2 k$ from the mean hidden state over, separately, the first third and the last third of each instrumented trajectory, at every probe layer, scored by leave-one-template-out cross-validation so a probe is never evaluated on a carrier sentence it was fitted on – solved in the dual ($n \approx 54$ items against up to 4096 features per layer) purely for tractability, with identical predictions to the primal. This is not an analogy to Theorem A(iii); a linear, cross-validated probe with a reported R^2 is exactly the readout class the theorem constrains, so the test evaluates the theorem's own claim directly. *Retention* is defined per model and family as the best-layer late-window R^2 divided by the best-layer early-window R^2: because the same probe class, the same items, and independently the best layer for each window are used at both ends, retention cancels probe capacity and item-difficulty confounds, leaving only the within-trajectory change. The matched control makes the comparison interpretable: the identical probe, the identical model, the identical layer-selection procedure, decoding the identical quantity from non-repetitive text of equal length.

Numbers. Table 11 reports retention for every model and condition in `data/results/probe.json`. Across the six true panel checkpoints, median retention on repeated text is 0.97 against 1.11 on controls, and repeated-text retention is lower than its own control in all six of six. Best-layer R^2 decoding $\log_2 k$ is modest even at its best: the panel median late-window best-layer R^2 on `word_rep` is ≈ 0.48 – well short of the near-perfect decodability [2] reports for text LMs – so the count is only ever coarsely recoverable here, at either end of the trajectory.

Table 11: Ridge-probe retention (late-window best-layer R^2 / early-window best-layer R^2), from `data/results/probe.json`. All entries n=54 items.

Model	`word_rep` retention	`control_word` retention
Llasa-1B	0.97	1.09
Llasa-3B	0.87	0.99
Llasa-8B	0.96	1.09
Qwen3-TTS-0.6B	1.46	1.74
Qwen3-TTS-1.7B	0.77	1.12
XTTS-v2	1.92	2.85
Panel median	**0.97**	**1.11**
XTTS-v2 (no penalty, abl., not pooled)	1.37	1.40

A second pooling caveat, found while verifying this section. The main paper's probe statistics (0.97 repeated, 1.12 control, "repeated lower in seven of 7 variants") are computed by `analysis/make_numbers.py` over every entry in `probe.json`, including `xtts2norp`. Unlike that same script's behavioural-dissociation aggregation, which explicitly filters `beh[~beh.model.isin(ABLATIONS)]` (Section 8.5), its probe-aggregation loop iterates `pr.values()` with no corresponding filter, so the reported "7" variants are the six true panel checkpoints plus the ablation counted as a nominal seventh – exactly the double-counting Section 6 and `findings.md`'s own "Lessons and Constraints" warn against elsewhere in this project. Recomputed over the six true panel checkpoints alone (Table 11), the medians are 0.97 repeated and 1.11 control – materially identical to the pooled-with-ablation figures – and repeated retention is still lower in six of six. The ablation's inclusion therefore does not manufacture this particular result; the correct fix is that `make_numbers.py`'s probe section should apply the same `ABLATIONS` filter

its behavioural section already applies, and the main paper's "seven of 7" should read "six of 6," with `xtts2norp` reported as a consistent seventh, separate data point rather than folded into the panel statistic. This document does not modify that script; it reports the discrepancy and gives the corrected, panel-only numbers above so that the supplementary record is self-consistent even where the generating script currently is not.

Conclusion. The representational-degradation account is favoured over the output-policy account, but only on qualified terms that this document states plainly rather than rounds up. On the affirmative side: retention is below 1 – within-trajectory loss – on repeated text in every panel checkpoint, and at or above 1 – flat or improving – on the matched control in five of six (Llasa-3B's control retention, 0.99, is essentially flat rather than clearly improving). On the qualified side: best-layer R^2 never approaches the near-perfect decodability the competing account reports for text LMs, so even early in generation the count is only coarsely present; and retention below 1 is degradation, not erasure – the probe still recovers a non-trivial signal late in most trajectories, so this test cannot rule out that some of the count remains linearly readable somewhere the aggregate effective-rank statistic of Section 8.3 would not detect. As the main paper's Discussion already states, the two accounts are not mutually exclusive, and this probe result is evidence for representational degradation over pure output-policy failure, not proof against the latter.

8.5 The XTTS repetition-penalty ablation

The objection. XTTS-v2 ships `repetition_penalty`= 5.0 on its acoustic-token decoding (Section 6), an aggressive built-in anti-repetition mechanism active during every sampling pass this study measures. A skeptical reading: XTTS-v2's comparatively high k^* among the panel is an artifact of this decoding-time crutch rather than evidence bearing on Theorem A at all, and any argument built on contrasting XTTS-v2 against the Llasa scale ladder – e.g. that capacity alone does not order k^* – is confounded by an uncontrolled decoding-rule difference between the two families.

What would have to be true. If the penalty, not the mechanism the paper argues for, were responsible for XTTS-v2's collapse-resistance, then disabling it should make the periodicity-vs-length *dissociation* disappear or shrink toward a decoding-rule artifact – the model should stop showing a repeated-vs-control gap once the crutch is removed, because there would be no underlying mechanism left for the gap to reflect.

Test design. The registry (Section 6, `src/common/registry.py`) carries `xtts2norp` as a seventh entry: the identical XTTS-v2 checkpoint, generated by `src/models/xtts_gen.py --repetition-penalty 1.0` (penalty disabled), scored and instrumented through the identical pipeline as every panel member, under a separate registry key so its outputs never mix with `xtts2`'s on disk. This is the right design for the objection specifically because it manipulates exactly one decoding-time knob while holding the checkpoint weights, the stimuli, and every other setting fixed – and because it is a within-model manipulation rather than a new independent system, it is excluded by construction from every panel-pooled statistic in this project (`ABLATIONS = {"xtts2norp"}`, enforced in `analysis/make_numbers.py`, `capacity.py`, `capacity_confound.py`, and `unit_invariance.py`, modulo the one gap in the probe aggregation noted in Section 8.4). Pooling it would silently inflate the panel size and double-count XTTS-v2's architecture – precisely the failure `findings.md`'s "Lessons and Constraints" records was caught and fixed once already in this project.

Numbers. With the penalty (`xtts2`): $k^* = 4$, accuracy at $k \geq 6$ is 13.9% repeated vs. 59.3% control. Without it (`xtts2norp`): $k^* = 2$, 1.9% repeated vs. 14.8% control. Disabling the penalty makes the model strictly worse at counting – k^* falls from 4 to 2 and both accuracies drop – while the

relative dissociation not only survives but widens: the control-to-repeated accuracy ratio is $\approx 4.3\times$ with the penalty and $\approx 7.8\times$ without it. On the capacity side (`data/results/capacity.json`), disabling the penalty compresses the entire dynamic range of the measurement: repeated-text gain falls from 21.6 [14.1, 28.7] to 7.7 [4.6, 10.8] and control gain falls from 36.1 [31.2, 40.9] to 8.7 [5.1, 12.4], so the ratio moves from 0.60 to 0.89 and the paired-gap confidence interval, which excludes zero with the penalty on ([6.0, 20.8]), crosses it with the penalty off ([−1.8, 6.3], `ci_separated: false`). This is the single checkpoint-level cell in the entire panel-plus-ablation set where the capacity contrast does not reach significance, and it is reported here as such rather than omitted.

Conclusion. The penalty is masking the collapse, not producing it: the model counts worse without it, and the periodicity-vs-length behavioural gap persists – indeed strengthens in relative terms – once the crutch is removed, which is the opposite of what the objection needs. XTTS-v2's place in the panel *with* the penalty is accordingly a conservative test of the theory, not an inflated one. The qualification is on the capacity side specifically: removing the penalty shrinks both conditions' dynamic range so much that the effective-rank contrast, though still pointed the same direction (ratio $0.89 < 1$), is no longer statistically resolved for this one ablated checkpoint. That instability is itself informative – it is consistent with the penalty's role being to inject additional acoustic-token diversity into *every* generation, repeated or not, which is exactly the kind of output-diversity confound Section 8.3 controls for directly on the unablated panel – but it means the ablation corroborates the behavioural dissociation more strongly than it corroborates the capacity measurement, and the two should not be conflated.

8.6 Statistical treatment

Wilson intervals for proportions. Every proportion reported as a percentage with an interval in the main paper and in this document – counting accuracy, the fraction of matched pairs with the control less probable, the fraction of repeated-vs-control paired differences that are positive – uses the Wilson score interval (`wilson` in `analysis/make_numbers.py`), not the normal approximation. The reason is not a stylistic preference: several of this project's cells sit at or near 0% or 100% (word-repetition accuracy at high k for the weaker checkpoints; the gpt2-large naturalness check at exactly 54/54, Section 8.1), where the normal interval's half-width $z\sqrt{p(1-p)/n}$ degenerates toward zero or produces bounds outside $[0, 1]$ that then have to be clipped by hand. Wilson's interval, derived by inverting the normal test for p rather than plugging $\hat{p}$ into a symmetric approximation around it, stays inside $[0, 1]$ by construction and remains informative at the boundary: the gpt2-large cell, 54/54, has an undefined normal interval (zero estimated variance) but a Wilson interval of [93.4, 100.0]%, which is the number actually reported.

Bootstrap over templates, not items. Every confidence interval on a *mean* or *slope* in this project – the capacity-gain intervals of Section 4, the naturalness-gap interval of Section 8.1 – resamples the finite set of stimulus *templates* with replacement, never the larger set of items or pairs built from them. The reason is structural, not a robustness nicety: the 54 naturalness pairs are 6 templates $\times$ 9 values of k, and the panel's capacity-gain fits are built from a handful of carrier sentences crossed with many k and (for the instrumented pass) one seed; items sharing a template share carrier syntax, word count, and, for word-repetition items, seven of eight possible filler choices, so they are correlated draws rather than independent ones. Resampling items directly would treat, e.g., the 54 naturalness pairs as 54 independent observations when the design in fact contains six independent units; the resulting interval would be too narrow, understating genuine uncertainty in exactly the comparisons this section exists to get right.

Per-cell sample sizes. This document reports n alongside every point estimate in Sections 8.1–8.5 rather than only in a methods paragraph, because several cells are close to the floor at

which an effect can be resolved at all: the diversity-adjusted capacity-confound test is estimable for only 3 of 6 panel checkpoints (Table 10); its correct-only variant has one cell at n=0 (Llasa-8B) and one at n=14 (Llasa-1B, repeated side) that produces a ratio (0.01) this document explicitly flags as unstable rather than load-bearing; the linear-probe retention statistic is computed from n=54 instrumented items per model and family throughout, the same instrumented subset documented in Section 3. Reporting these n's is what makes it possible to tell an unstable small-sample cell (Llasa-1B's correct-only ratio) from a stable one (the panel medians, which agree to within two hundredths across three different controls) without re-deriving the analysis from source.

No multiple-comparison correction, stated so a reader can apply their own. Nowhere in this project's analysis pipeline – confirmed by reading every script cited in this section – is a Bonferroni, Holm, or Benjamini–Hochberg correction applied across the per-model tests that make up the panel-level claims. The count a reader wanting a family-wise guarantee would need depends on which family of tests they consider the relevant unit. The primary capacity-gain contrast of Section 4.3 (main paper) is six per-model disjoint-interval tests, one per panel checkpoint (`ci_separated` in `capacity.json`). The confound checks of Section 8.3 add up to two further ratio estimates per model (diversity-adjusted, correct-only) on top of the raw one, for as many as eighteen nominal model-level cells across the three variants, of which fourteen are actually estimable (6 raw + 3 adjusted + 5 correct-only, Table 10). The probe-retention comparison of Section 8.4 is six per-model directional tests (repeated retention below control retention). The naturalness by-k breakdown of Section 8.1 is nine k-values reported under two mutually scrutinised scorers. None of these is adjusted for having been looked at together. A reader who wants a Bonferroni-style family-wise error rate should divide the nominal $\alpha = 0.05$ by six for the primary capacity contrast alone, or by the low twenties if every per-model cell across Sections 8.3 and 8.4 is treated as one family; this document supplies the counts rather than picking a family on the reader's behalf, because the right grouping depends on which claim in the main paper the reader is trying to stress-test.

References

[1] I. Viakhirev, K. Borodin, and G. Mkrtchian, "From dispersion to attraction: Spectral dynamics of hallucination across whisper model scales," *arXiv preprint arXiv:2604.08591*, 2026.

[2] S. Venkatesh, "Repeated-token counting reveals a dissociation between representations and outputs," *arXiv preprint arXiv:2605.09239*, 2026.